CS725PE: SOFTWARE PROCESS & PROJECT MANAGEMENT (Professional Elective - V)

UNIT - I

Software Process Maturity

Software maturity Framework, Principles of Software Process Change, Software Process Assessment, The Initial Process, The Repeatable Process, The Defined Process, The Managed Process, The Optimizing Process.

Process Reference Models

Capability Maturity Model (CMM), CMMI, PCMM, PSP, TSP).

SOFTWARE PROCESS MATURITY – INTRODUCTION

Software Process:

Software process (also knows as software methodology) is a set of related activities that leads to the production of the software. These activities may involve the development of the software from the scratch, or, modifying an existing system.

Any software process must include the following four activities that are fundamental to software engineering.

1. **Software specification** (or requirements engineering): The functionality of the software and constraints on its operation must be defined.
2. S**oftware design and implementation**: The software to meet the specification must be produced.
3. **Software verification and validation**: The software must be validated to ensure that it does what the customer wants.
4. **Software evolution** (software maintenance): The software must evolve to meet changing customer needs.

In practice, they include sub-activities such as requirements validation, architectural design, unit testing …etc.

There are also **supporting activities** such as configuration and change management, quality assurance, project management, user experience.

Along with **other activities** aim to **improve** the above activities by introducing new techniques, tools, following the best practice, process standardization (so the diversity of software processes is reduced), etc.

When we talk about a process, we usually talk about the activities in it. However, a process also includes the process description, which includes:

1. **Products**: The outcomes of the activity. For example, the outcome of architectural designs maybe a model for the software architecture.

2. **Roles**: The responsibilities of the people involved in the process. For example, the project manager, programmer, etc.

3. **Pre and post conditions**: The conditions that must be true before and after an activity. For example, the pre condition of the architectural design is the requirements have been approved by the customer, while the post condition is the diagrams describing the architectural have been reviewed.

Software process is complex, it relies on making decisions. There's no ideal process and most organizations have developed their own software process.
For example, an organization works on critical systems has a very structured process, while with business systems, with rapidly changing requirements, a less formal, flexible process is likely to be more effective.

1) A SOFTWARE MATURITY FRAMEWORK

Background Software process encompasses the set of tools, methods, and practices used to produce a software product
Objectives (done simultaneously)
- Produce products according to plan
- Improve the organization's capability to produce better products

Basic principles: Statistical process control and predictable performance
The foundation of statistical control is measurement

1. Software Process Improvement

An important first step in addressing software problems is to treat the entire software task as a process that can be controlled, measured and improved. To improve s/w capabilities, organizations must take 6 steps:

1. Understand the current status of their development process or processes.
2. Develop a vision of the desired process.
3. Establish a list of required process improvement actions in order of priority
4. Produce a plan to accomplish the required actions.
5. Commit the resources to execute the plan
6. Start over at step 1

2. Process Maturity Levels → 5 levels of process maturity

1. **Level 1 – Initial**: Until the process is under statistical control, orderly progress in process improvement is not possible. While there are many degrees of statistical control, the first step is to achieve rudimentary predictability of schedules and costs.

2. **Level 2 – Repeatable**: The organization has achieved a stable process with a repeatable level of statistical control by initiating rigorous project management of commitments, costs, schedules and changes.

3. **Level 3 – Defined**: The organization has defined the process as a basis for consistent implementation and better understanding. At this point advanced technology can usefully be introduced.

4 **Level 4 – Managed**: The organization has initiated comprehensive process measurements and analysis. This is when the most significant quality improvements begin.

5. **Level 5 – Optimizing**: The organization now has a foundation for continuing improvement and optimization of the process.

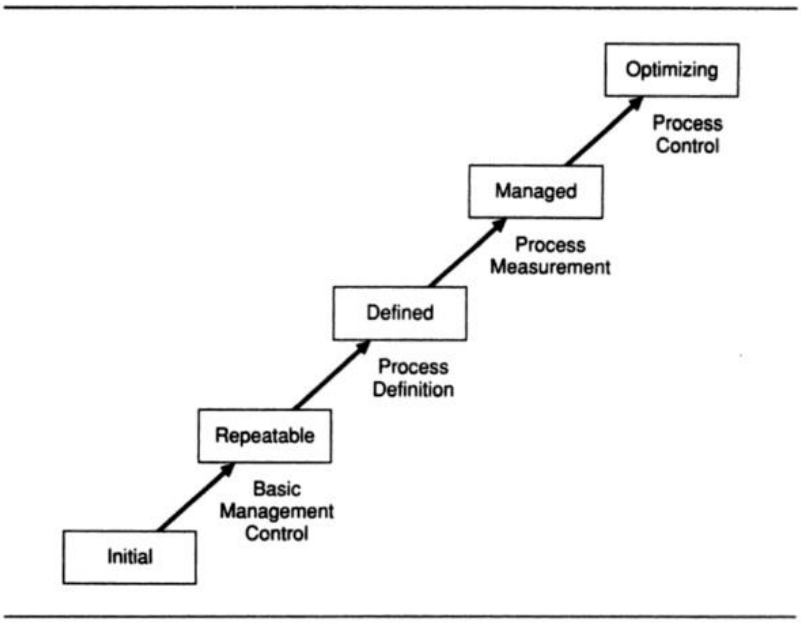


FIGURE 1.1
Process Maturity Levels

These Levels have been selected because they:

- Reasonably represent the actual historical phases of evolutionary improvement of real software organizations
- Represent a measure of improvement that is reasonable to achieve from the prior level
- Suggest interim improvement goals and progress measures
- Make obvious a set of immediate improvement priorities, once an organizations status in this framework is known.

This process maturity structure is intended for use with an assessment methodology and management system.
Assessment methodology helps an organization identify its specific maturity status
Management system establishes a structure for implementing the priority improvement actions

The Initial Process (Level 1)

Usually ad hoc and chaotic - Organization operates without formalized procedures, cost estimates, and project plans. Tools are neither well integrated with the process nor uniformly applied. Change control is lax, and there is little senior management exposure or understanding of the problems and issues. Since many problems are deferred or even forgotten, software installation and maintenance often present serious problems.
While organizations at this level may have formal procedures for planning and tracking work, there is no management mechanism to insure they are used. Procedures are often abandoned in a crisis in favour of coding and testing. Level 1 organizations don't use design and code inspections and other techniques not directly related to shipping a product.

Organizations at Level 1 can improve their performance by instituting basic project controls. The most important ones are
• Project management
• Management oversight
• Quality assurance
• Change control

The Repeatable process (Level 2)

This level provides control over the way the organization establishes plans and commitments. This control provides such an improvement over Level 1 that the people in the organization tend to believe they have mastered the software problem. This strength, however, stems from their prior experience in doing similar work. Level 2 organizations face major risks when presented with new challenges.

Some Major Risks

- Unless they are introduced with great care, new tools and methods will affect the processes, thus destroying the historical base on which the organization relies.
- When the organization must develop a new kind of product, it is entering new territory.
- Major organizational changes can also be highly disruptive

Key actions required to advance from Repeatable to the next stage, the Defined Process establish a process group:
Establish a process group: A process group is a technical resource that focuses heavily on improving software processes. In most software organizations, all the people are generally devoted to product work. Until some people are assigned full-time to work on the process, little orderly progress can be made in improving it.
Establish a software development process architecture (or development cycle): that describes the technical and management activities required for proper execution of the development process. The architecture is a structural decomposition of the development cycle into tasks, each of which has a defined set of prerequisites, functional decompositions, verification procedures, and task completion specifications.
Introduce a family of software engineering methods and technologies: These include design and code inspections, formal design methods, library control systems, and comprehensive testing methods. Prototyping and modern languages should be considered.

The Defined Process (Level 3)

The organization has the foundation for major and continuing change. When faced with a crisis, the software teams will continue to use the same process that has been defined.
However, the process is still only qualitative; there is little data to indicate how much is accomplished or how effective the process is. There is considerable debate about the value of software process measurements and the best one to use.

The key steps required to advance from the Defined Process to the next level are

• Establish a minimum set of basic process measurements to identify the quality and cost parameters of each process step.
• Establish a process database and the resources to manage and maintain it.
• Provide sufficient process resources to gather and maintain the process data and to advise project members on its use.
• Assess the relative quality of each product and inform management where be done by an independent quality assurance group

The managed Process (Level 4)

Largest problem at Level 4 is the cost of gathering data. There are many sources of potentially valuable measure of the software process, but such data are expensive to collect and maintain.

Productivity data are meaningless unless explicitly defined. For example, the simple measure of lines of source code per expended development month can vary by 100 times or more, depending on the interpretation of the parameters.

When different groups gather data but do not use identical definitions, the results are not comparable, even if it makes sense to compare them. It is rare when two processes are comparable by simple measures. The variations in task complexity caused by different product types can exceed five to one. Similarly, the cost per line of code for small modifications is often two to three times that for new programs.

Process data must not be used to compare projects or individuals. Its purpose is too illuminate the product being developed and to provide an informed basis for improving the process. When such data are used by management to evaluate individuals or terms, the reliability of the data itself will deteriorate.
The two fundamental requirements for advancing from the Managed Process to the next level are:
• Support automatic gathering of process data. All data is subject to error and omission, some data cannot be gathered by hand, and the accuracy of manually gathered data is often poor.

• Use process data to analyze and to modify the process to prevent problems and improve efficiency.

The Optimizing Process (Level 5)

To this point software development managers have largely focused on their products and will typically gather and analyze only data that directly relates to product improvement. In the Optimizing Process, the data are available to tune the process itself.

For example, many types of errors can be identified far more economically by design or code inspections than by testing. However, some kinds of errors are either uneconomical to detect or

almost impossible to find except by machine. Examples are errors involving interfaces, performance, human factors, and error recovery.

So, there are two aspects of testing: removal of defects and assessment of program quality. To reduce the cost of removing defects, inspections should be emphasized. The role of functional and system testing should then be changed to one of gathering quality data on the program. This involves studying each bug to see if it is an isolated problem or if it indicates design problems that require more comprehensive analysis.

With Level 5, the organization should identify the weakest elements of the process and fix them. Data are available to justify the application of technology to various critical tasks, and numerical evidence is available on the effectiveness with which the process has been applied to any given product.

3. People in the Optimizing Process

- The optimizing process enhances the talents of quality people of several ways. It helps managers understand where help is needed and how best to provide people with the support they require.
- The optimizing process provides a disciplined environment for professional work.
- Discipline is required in large software projects to ensure that many people involved use the same conventions, don't damage each other's products.

4. The need for the optimizing process

The optimizing process provides the foundation for significant advances in software quality and simultaneous improvements in productivity.

There is an urgent need for better and more effective software organizations.

To meet this need, software managers and professionals must establish the goal of moving the optimizing process.

2. THE PRINCIPLES OF SOFTWARE PROCESS CHANGE

It describes the principles for changing the software process and discusses some common misconceptions about software work.

Software process change rests on many of the principles of organizational change

However, focusing only on talent can lead into a blind alley –The best people are always in short supply –You probably have the best team you can get right now –With proper leadership and support, most people can do much better work than they are currently doing now

1. Process in perspective

Two key areas that need to be considered are people and design methods

People

Better people clearly do better work; we want best team for our project; focusing only on talent

• The best people are always in short supply
• You probably have about the best team you can get right now.
• With proper leadership and support, most people can do much better than they are currently doing.

Design

• Superior products have superior design.
• Successful products are designed by people who understand the application (domain engineer).
• A program should be viewed as executable knowledge
• Program designers should have application knowledge.

2. The Six Basic Principles

The basic principles of s/w process change are:

- **Major changes to the software process must start at the top**: Senior management leadership is required to launch the change effort and to provide continuing resources and priority.
- **Ultimately, everyone must be involved**: S/w engineering is a team effort, and anyone who does not participate in improvements will miss the benefits and may even inhibit progress
- **Effective Change requires a goal and knowledge of the current process**
- **Change is continuous**: S/w process improvement is not a one shot effort; it involves continual learning and growth.
- **Software process changes will not be retained without conscious effort and periodic reinforcement**
- **Software process improvement requires investment**: It takes planning, dedicated people, management time and capital investment

▪ Major changes to the software process must start at the top:

Major changes require leadership. Making changes calls for a management team with the conviction that long term improvements are both possible and essential. These managers must then set challenging goals, monitor progress, and insist on performance.

▪ Ultimately, everyone must be involved:

With an immature s/w process, the software professionals are forced o improvise solutions. In a s/w process people are the most important ingredient. It is essential to recognize their desire to do good work. The need is thus to focus on repairing the process and not the people.

- **Effective Change is built on knowledge:**

An effective change program requires a reasonable understanding of the current status. An assessment is an effective way to gain this understanding. Assessment is a learning process, aimed at gaining a clear definition of the key problems and the professionals views of what should be done to resolve them. Without such knowledge priorities are the most important part of an improvement program. S/w professionals need most help in controlling requirements, coordinating changes, managing plans, managing interdependencies and coping with system design issues.

- **Change is continuous**

Human intensive processes are never static. In Dealing with dynamics, 3 points are important:

1. Reactive Changes generally make things worse
2. Every defect is an improvement opportunity
3. Crisis prevention is more important than crisis recovery.

While process can be improves under pressure, doing so is often a mistake. In a crisis, the focus is on quickly fixing the immediate problem, not on improving the process. While temporary process patches may be needed, it is wise to make permanent process changes in a more disciplined way.

- **Software process change won't stick by themselves**

The tendency for improvements is best characterized by the physics term *entropy*

It refers the steady increase in the randomness or disorder of physical processes

Precise and accurate work is so hard that it is rarely sustained for long without reinforcement. This means that new methods must be carefully introduced and periodically monitored, or they will rapidly decay.

Human adoption of new process methods involves 4 stages:

1. Installation: This involves initial installation of the methods and training in their proper use.
2. Practice: Here the people learn to perform the methods as instructed.
3. Proficiency: Next comes the traditional learning curve, as people gradually improve their efficiency.
4. Naturalness: Finally the methods are so ingrained that they are performed without intellectual effort.

- **It takes time, skill and money to improve the software process.**

The key points are:

1. To improve the software process, someone must work on it
2. Unplanned process improvement is wishful thinking

3. Automation of a poorly defined process will produce poorly defined results
4. Improvements should be made in small, tested steps.
5. Train, train, train!

3. Common Misconceptions about the software process

Effective change depends on realism. Some of the more common software process misconceptions are:

- ***We must start with firm requirements.*** There is a widespread but fallacious view that requirements are the customers' job and that development should not start until they are explicitly defined.
- ***If it passes test, it must be OK.*** If the generally dismal record of software quality problems doesn't prove this false, nothing will.
- ***Software quality can't be measured.*** While not as well recognized, this too is false.
- ***The problems are technical.*** In spite of the many improved languages, tools, and environments, the problems of software cost, schedule, and quality remain.
- ***We need better people.*** Since the software professionals make the errors, some people erroneously feel that they should be blamed for them.
- ***Software management is different.*** While this is a new and unique field, traditional management methods can and should be used.

i. We must start with firm requirements

A software perversity law seems to be the more firm the specifications, the more likely they are to be wrong. With rare exceptions, requirements change as the software job progresses. Just by writing a program, we change our perceptions of the task. Requirements cannot be firm because we cannot anticipate the ways the tasks will change when they are automated.

For large-scale programs, the task of stating a complete requirement is not just difficult; it is impossible. Generally, we must develop software incrementally.

Stable requirements

However, we must have stability long enough to build and test a system. This means the incremental development process must start with minimal requirements that we and the user "know" are valid. For large or integrated systems, the system design organization must also be part of the process

Premature requirement freezes

However, if we freeze requirements too early, later retrofits are expensive and time-consuming. The requirement-based process motivates counterproductive behavior. With software challenge we balance the unknown nature of the requirements with the business need for a firm contractual relationship

ii. If it passes Test, it must be OK

The converse is certainly true: If it doesn't pass test, it isn't OK! As we build larger and more complex programs and their operating environments become more complex and unpredictable, testing is a progressively less reliable indicator of system quality. The reason is that many design and coding errors are caused by the programmer failing to anticipate some improbable condition. As the number of possible program conditions increases, the feasibility of exhaustively testing thus declines.

If we cannot completely test programs, how can we handle complex systems? The answer is that we can do some things with reasonable assurance, but not everything. While the balance of this book describes some approaches to this problem, a few simple ground rules will help:

- Establish as high-quality a software process as you can.
- Determine the most complex, essentially error-free programs you can build with this process.
- When highly reliable programs are needed, limit their complexity to what you know you can produce with your essentially defect-free process.
- Test your programs as thoroughly as you economically can.
- Recognize that your capability to build progressively larger defect-free programs will improve as you manage and improve your software process.

iii. Software Quality can't measured

There is no measure that comprehensively represents programming quality. On the other hand, such comprehensive measures don't exist for other complex products either.

With today's software, quality improvement is generally governed by the amount of effort we can afford to spend on quality improvement.

The initial step must be to define the purpose for making software quality measurements.

iv. The problems are technical

There are many fascinating and important technical software problems.

Their highest priority is to establish an orderly process that will permit their people to operate efficiently.

Low-maturity software organizations spend most of their energy in controlling the high volume of changes, recovering from late-discovered defects, and reacting to unanticipated crises. While such organizations generally produce products, their costs are rarely predictable and their quality is often marginal. Their major problems are not the lack of adequate languages, reuse methods, advanced tools, or integrated environments.

v. We need better people

I once heard a senior executive seriously propose that we find the programmers who made the errors and get rid of them.

We must certainly get the most energetic and best-qualified people we can, but this alone will not solve all the problems. Most of the problems can only be fixed through management action.

vi. Software management is different

- Software is generally more complex than other engineering products.
- Since software engineering is relatively new, there are not yet many managers or senior professionals with sufficient experience to appreciate the benefits of an effective process.
- In other engineering fields, release to manufacturing provides a natural discipline that is not present with software.
- The software discipline is not grounded in natural science.
- Software is often the system element that presents the function to the end user.
- Software also is often the crucial ingredient that couples all the other system elements together.
- Software is often viewed by non–software practitioners as a black art.

4. A Strategy for implementing Software Process change

Apply three phases: unfreeze, move, refreeze
Unfreeze by identifying champions, sponsors, and agents
–Champions initiate the change process
–Sponsors are the senior managers
–Agents lead change planning and implementation
Move by using key elements of effective change: planning, implementation, and communication
Refreeze to ensure that an achieved capability is retained in general practice

a) Champions, Sponsors, and Agents

• **Champions** - Ones who initiate change. They bring management's attention to the subject, obtain the blessing of a sponsor, and establish the credibility to get the change program launched. The champion maintains focus on the goal, strives to overcome obstacles, and refuses to give up when the going gets tough.
• **Sponsors** - Senior manager who provides resources and official backing. Once a sponsor is found, the champion's job is done; it is time to launch the change process.

• **Agents** - Change agents lead the planning and implementation. Agents must be enthusiastic, technically and politically savvy, respected by others, and have management's confidence and support.

b) Elements of Change

The key elements of effective change are

• Planning
• Implementation
• Communication

c) Refreezing

It can take many forms, but the basic objective is to ensure that an achieved capability is retained in general practice

Several common techniques

- Retain the management team that instituted the change.
- Modify the organization's procedures.
- Establish measurements and incentives.
- Set up a dedicated staff to monitor and support performance.
- Establish an education and training program.

3. SOFTWARE PROCESS ASSESSMENT

Process assessment helps s/w organizations improve themselves by identifying their critical problems and establishing improvement priorities. The basic assessment objectives are:

- To learn how the organization works.
- To identify its major problems
- To enroll its opinion leaders in the change process

A s/w process assessment is not an audit but a review of a s/w organization to advice its management and professionals on how they can improve their operation. The purpose of an assessment is to identify the highest priority areas for improvement and to provide guidance on how to make those improvements.

The essential approach is to conduct a series of structured interviews with key people in the organization to learn their problems, concerns, and creative ideas.

1. Assessment Overview

A software process assessment is not an audit but a review of a software organization to advise its management and professionals on how they can improve their operation.

The purpose of an assessment is to identify the highest-priority areas for improvement and to provide guidance on how to make those improvements.

Assessments are based on the principle that the local managers and professionals want to improve their own operation and that their primary need is guidance on what to do and how to do it.

2. Assessment Phases

Assessments are in 3 phases: Preparation, assessment and recommendations.

• Preparation - Senior management agrees to participate in the process and to take actions on the resulting recommendations or explain why not. Concludes with a training program for the assessment team

• Assessment - The on-site assessment period. It takes several days to two or more weeks. It concludes with a preliminary report to local management.

• Recommendations - Final recommendations are presented to local managers. A local action team is then formed to plan and implement the recommendations.

3. Five Assessment Principles

The basic requirements for a good assessment are a competent team, sound leadership, and a cooperative organization.

Some special considerations should be kept in mind they are:

- The need for a process model as a basis for the assessment
- The requirement for confidentiality
- Senior management involvement
- An attitude of respect for the views of the people in the organization being assessed
- An action orientation

1. Start with a process model - Without a model, there is no standard; therefore, no measure of change.

An assessment implies a standard. The organization's process is reviewed in comparison with some vision of how such processes should be performed. This, of course, is the crux of the assessment process.

To avoid these problems, it is wise to base an assessment on a common view of the desired software process. Such a model provides a basis for orderly exploration as well as a framework for establishing problem priorities. With such a focus, the entire team can work together on the key issues and recommendations.

2. Observe strict confidentiality - Otherwise, people will learn they cannot speak in confidence. This means managers can't be in interviews with their subordinates.

Confidentiality permits the assessors to talk to people at all levels of the organization. If the managers suspect that the findings will be passed to higher management, they will properly insist on being present at every interview. Unfortunately, when managers are present, professionals rarely say anything that their managers don't know already or with which they might disagree. There is then no reason to have an assessment. The managers could present this official view far more efficiently in a two-hour briefing.

Confidentiality is required at all organizational levels. The professionals must know that their comments will not be attributed to them. Several projects should be reviewed at once, and the project managers should be told that the results for their projects will be given only to them. Site management is then provided a composite picture of the overall facility. This ensures that no single project or individual is identified with any specific problem.

3. Involve senior management

The senior manager sets the organization's priorities. This local manager typically gives final approval for software commitments and answers to corporate management when things go wrong. While some senior managers are responsible for multiple projects in several locations, it is wise to focus in a reasonably local geographic area. This focus minimizes project disruption, simplifies assessment arrangements, and generally facilitates subsequent action planning and implementation. In the following discussions, the senior manager of this total organization will be called the *site manager*.

The site manager must be personally involved in the assessment and its follow-up action plans. If not, the work will not be given sufficient priority. While some initial good-faith attempts may be made, the first crisis will soon preempt these process improvement efforts. If the assessment is to have any lasting impact, the site manager must personally participate, assign qualified people, and periodically review the progress of the resulting action plans.

Without this support the assessment will likely be a waste of time. The lower-level people can generally handle their routine problems, but lasting improvements must survive the periodic crises. That is when the process is under most stress, when management is most likely to defer nonessential work, and when serious disasters are most likely. Since software crises are common, if the site manager won't protect the process improvement efforts, they will not likely continue long enough to do much good.

4. Keep an open mind and a level head

Any assessment can easily seem arrogant. A group of remote "experts" reviews a large and complex organization and in a few days tells them what they are doing wrong and what they should do to improve. Generally the local people work hard, are dedicated to doing a good job, and are trying to improve. They are thus properly skeptical of any brief study and doubt it can have any lasting impact.

The local professionals will soon sense if an assessment team arrives thinking it has all the answers. Their natural reaction will be to show these "experts" they are not so smart after all. This leads to an unspoken wish that the assessment will fail. Under these conditions, it often will.

This distrust of outside experts is not only understandable but is quite proper. A small team of outside experts cannot hope to identify in a few days the most critical problems in any organization. Complex problems rarely have simple answers, and the subtleties of most organizations are far too intricate for any group to fathom quickly.

The fundamental assumption must be that the on-site professionals are smart, motivated, and have many good ideas. If they can be convinced to share their knowledge, the assessment can be a catalyst for self-improvement. The professionals will be willing to share, however, only when they see the assessment as a way to get help rather than as a threat of exposure.

A highly critical attitude or a lack of interest in local views by the assessment team can be deadly. When good work is found, it should be recognized and identified so other groups can take advantage of it. Surprisingly, for each software problem there is often someone in the organization who has already solved it. Making this capability visible can be one of the greatest and most immediate benefits of the assessment. Mistakes and oversights must also be identified, but they should be objectively reported without attribution, criticism, or blame. The team must recognize that its suggestions are only ideas that will have to be evaluated and adjusted to local conditions. As difficult as it is to achieve, the proper attitude is one of open-minded and supportive professionalism.

Even when the assessment team is appropriately supportive, some local people will be resentful and not cooperate. If the team members' actions clearly demonstrate their desire for active collaboration with the on-site professionals, however, this will be recognized and people generally will respond positively.

5. Focus on Action

Finally, to have lasting effect, the assessment must be directed toward improvement. An action orientation keeps the questions focused on current problems and the need to solve them. If the assessment turns into a general exploration, it will not focus on the priority issues or produce recommendations that will be implemented.

An aborted or misguided assessment will do little good and can even make the situation worse. Prior to an assessment the professionals generally are aware of their worst problems and often assume management is not. While this leads them to view management as mildly inept, they frequently assume management doesn't understand the issues and cannot be expected to solve them. After an assessment, this is no longer the case. An expert study has heard their concerns and suggestions for what should be done about them. These results are then reported to the site manager. After all this, any manager who does not take action will be seen by the people as either incompetent or unconcerned with their problems. In either case their morale will suffer. In net, management must either focus on taking action or not do an assessment.

4. The Assessment Process

The first step in any assessment is to identify the organization to be assessed and the team to do it. This typically requires the site manager's commitment to doing the assessment and willingness to assign sufficient skilled resources to get it done.

1. Forming an Assessment Team:

The assessment team leader is selected first. This is someone who has considerable s/w experience, has the ability to lead small groups, and is able to convincingly present the results. This leader should have assessment experience or should obtain advice and assistance from someone who has.

The assessment team members should all be experienced s/w developers, and one or more should have experience in each phase of the software process. Four to six persons form an a team, although more can be used if desired. Since larger teams cost more money and are harder to manage, an upper limit of 8 to 10 participants is usually wise.

The team members should be drawn from several groups within the organization being assessed and where possible most should come from projects other than those selected for review. Nobody should participate in the assessment who is otherwise personally involved in reviewing, supporting, or managing the projects being assessed.

2. Self-Assessment Considerations:

While it is possible for organizations to assess themselves, they should be aware of several potential problems. Site managers who desire a self assessment should thus reach agreement with their line managers on its importance and their commitment to support it. They must plan to assign people where needed, commit to attend the necessary meetings, and agree to participate in developing action plans to address the recommendations.

GUIDELINES FOR SELECTING ASSESSMENT TEAM MEMBERS

Each assessment team member should:

- Have at least eight to ten years professional software experience
- Be well respected in the organization
- Be able to deal with people in an informal and nonthreatening manner
- Be a team player
- Have attended assessment training with this team

No assessment team member should:

- Be currently serving in an audit or review capacity for any of the projects being assessed
- Be a line manager over any of the projects being assessed or people being interviewed
- Be working directly on any of the projects being assessed or working on their direct support

3. Assessment Ground Rules:

It is desirable to have a written set of assessment ground rules for the organization being assessed and for the assessment team members.

The items to be covered in such an agreement are:

- The assessment results will be kept confidential by the assessment team members.
- The site manager personally agrees to participate in the opening and closing assessment meetings.
- In addition to the regular members, the site manager agrees to assign one or two local professional to handle the assessment arrangements and to lead the follow-up action plan work. They will be full assessment team members.
- The site manager commits to developing and implementing appropriate action plans in response to the assessment recommendations.
- The site manager agrees to designate a person responsible for developing the action plans. If possible this person should be a member of the assessment team.

4. Assessment Team Training:

As the assessment team is formed, the members must agree to participate fully during the training period, on-site review and wrap up meetings. The team leader conducts two-or three day training program for the entire assessment team. This program familiarizes the team members with the assessment process and helps to build a cohesive working group. A typical training program includes the following steps:

1. The assessment schedule and objectives are outlined.
2. The assessment principles are reviewed, together with the s/w process model used as the assessment framework.
3. The organization members briefly outline the organizations mission, its management structure, and its recent history.
4. The assessment guidelines are discussed and all team members are asked to sign the written agreement.
5. A team-building exercise is conducted to assist the group in developing an effective and mutually supportive mode of operation

6. The detailed plan for assessment period is covered including topics for each session the discussion leader assignments, who will note the findings and how, when and how team conclusions are to be reached, by whom and when any reports and presentations are to be prepared and presented.
7. When necessary, portions of this process are rehearsed until all members are comfortable with their roles.
8. On-site planning

5. The on-site period

The assessment starts with a presentation to the site manager and staff.

TYPICAL SEI-ASSISTED ASSESSMENT SCHEDULE

Day One

Assessment Overview (site manager, staff, participants)
Purpose and conduct of the assessment
Briefing (participants)
A detailed review of the assessment schedule
Questions and answers
Questionnaire (project representatives)
The questionnaire is completed for each project
Project Discussions (project representatives)
Each project representative separately meets with the assessment team to clarify any questions and be told of any materials that the team desires to discuss on day three

Day Two

Functional Area Interviews (functional representatives)
A series of meetings, each with four to six professionals who are experts in selected technical areas
Preliminary Findings (assessment team)
The assessment team formulates its preliminary findings

Day Three

Project Discussions (project representatives)
Each project representative separately meets with the assessment team to discuss the points needing clarification and to comment on the preliminary findings
Findings Formulation (assessment team)
The assessment team formulates its findings

Day Four

Findings Dry Run (assessment team)
The findings presentation is reviewed by the assessment team
Findings Review (project representatives)
The findings presentation is reviewed with the project representatives
Findings Presentation (site manager, staff, participants)
The assessment findings are presented to the site manager, staff, and all the people who participated in the assessment
Senior Management Meeting (site manager, assessment team leader)
A discussion of any special management considerations that were not covered during the findings presentation, the next steps required, and the need to start planning for action team formation
Assessment Postmortem (assessment team)
The assessment team reviews the assessment process and identifies any changes needed to improve it

5. Assessment Conduct

While most technical people enjoy discussing the products they are developing, such discussion rarely provides much insight into the organization's problems. The objective is to explore the implementation of real projects rather than the products being built or the way the work is supposed to be done. While an idealized composite of the best instances of project work might be interesting, it does not provide a useful basis for process improvement. The assessment should thus focus on what the projects actually do, how they do it, the problems encountered, and the results obtained.

A selected set of questions should be prepared in advance of the actual assessment period.

Meetings are next held with small groups of selected professionals who have expertise in various facets of the software process (see Table 3.3). In free-form discussions the professionals offer their views and suggestions on the key problems. The discussions should typically end with a question such as: "If you could fix one facet of the process, what would you do and why?" With most groups this will generally elicit several creative ideas.

GUIDELINES FOR SELECTING FUNCTIONAL AREA REPRESENTATIVES

Six to eight representatives are identified for each of several meetings of 90 to 120 minutes each. Potential areas of specialty for each meeting are:

- Quality Assurance and release
- Software integration and test
- Coding and unit test
- Requirements and design

Each of these representatives should be:

- A recognized opinion leader in the selected area
- Actually working on projects rather than on a staff
- A technical professional and not a manager
- Aware of the assessment process and the confidentiality arrangements

1. Probing Questions:

In conducting assessments it is hard to obtain really accurate information. The reasons for this are:

- *Questions are often misunderstood.* The English language is imprecise, and brief questions are invariably subject to several interpretations.

- *The respondents may have a different understanding of some common terms.* For example, discussion is often required to reach common understanding on the meaning of the terms "high-level language," "review," or "reusability."

- *The respondents may not be broadly aware of the work in their own organization.* Some professionals are narrowly focused on their specialty areas. Outside this sphere, they may be uninformed or even misinformed. Managers typically have a broader view, but their hands-on experience is sometimes dated and their current knowledge is often filtered by their people.
- *Occasionally people are unwilling to risk the truth.* While it is rare for someone to tell an out-and-out lie, stories can generally be couched in favorable terms and the "unusual events" that "really weren't representative" can easily be overlooked.

2. Assessment Conclusion:

At the assessment conclusion, the team prepares a report on its initial findings. The report should be a composite of site status, together with more detailed findings in key areas. Prior to reviewing this material with the site manager, the team should review it with the project managers. This should identify any overlooked problems.

GUIDELINES FOR FINDINGS FORMULATION

The findings should be limited to the top 10 to 12 items that are:
- **Major issues for most of the projects reviewed**
- **Key issues for advancing to the next maturity level**
- **Supported by evidence from the assessment**
- **Addressable by an action recommendation**
- **Specific—avoid sweeping generalizations**

3. Assessment Report:

The final assessment team action is the presentation of a written final report and recommendations to the site manager and staff.

A written assessment report should always be prepared because:

i) Writing the actual recommendations helps the assessment team understand precisely what it is recommending.

ii) Since presentations are generally worded, their interpretation is highly dependent on the listener's background and biases.

iii) A written report provides an idea vehicle for informing the professionals about what was found and recommended.

A TYPICAL OUTLINE OF THE FINDINGS PRESENTATION

Presentation Overview:
- Agenda
- Scope—projects assessed
- Assessment conduct—schedule
- Summary composite status—the bad news
- Strengths noted—it isn't all bad
- Findings summary
- Findings—each finding (see below)
- Next steps

Findings format:
- Finding—what the team observed
- Consequences—the implications of the finding for the organization
- Examples—specific instances of the finding (without identifying people or projects)

4. Action Plans:

The action plans are next prepared by the local site organization, generally under the guidance of the team member named for this purpose. If properly chosen, this member is now fully knowledgeable on the issues an is able to start quickly.

6. Reassessments:

Organizations should generally conduct follow-up assessments one to two years after the initial action plans have been developed and approved. This is improvement for several reasons:

- To assess the progress that has been made.
- To provide a visible future milestone for completion of the actions from the prior assessment
- To establish new priorities for continued improvements.

6. Implementation Consideration

The greatest assessment risk is that no significant improvement actions will be taken. Without proper management focus a few superficial efforts may be made, but soon everything will revert to business as usual.

Risks

Some of the other key risks and potential actions to alleviate them are schedule conflicts, inadequate support, and lack of follow-through.

Schedule Conflicts. Despite the best intentions, crises that conflict with assessment plans often arise. The most damaging of these require the site manager to miss either the opening or the closing meeting or both. Such unfortunate schedule conflicts have happened in about one-third of the assessments

Inadequate Support. In the few cases of inadequate management support that I have experienced, the reason was that the assessment commitment was made at too low a management level.

Lack of Follow-through. Frequently management changes or other high-priority issues reduce the focus on action plan implementation. In at least half of the organizations I have assessed, the site manager changed between the final assessment report and action plan completion.

Staffing

Staffing is generally the most serious implementation problem. In addition to staffing the assessment itself, the other staffing needs will generally include:

- A small, full-time staff to focus and to guide the improvement efforts—
- Part-time project participation in the action plan working groups
- Project review and implementation of the resulting actions

4. Initial Process

Most software organizations operate in the Initial Process Level at least some of the time; many organizations never leave it. Though it is so common, this chaotic state is as hard to describe as darkness; it is easier to point out what it lacks. The Initial Process is the lack of a managed, defined, planned, and disciplined process for developing software. Since there can be many degrees to this lack, there are obviously many gradations to the Initial Process. Table 4.1 summarizes the basic characteristics of each process level and the actions required to improve to the next higher maturity level.

1. The nature of the initial process

In the Initial Process, professionals are driven from crisis to crisis by unplanned priorities and unmanaged change. Such groups seldom meet commitments. They may deliver on schedule occasionally, but this is the exception and normally due to herculean individual efforts rather than the strength of the organization. Senior management makes unreasonable demands while cutting resources. Schedules are the only priority, but when they do deliver, nobody likes the results. Many first-class professionals leave as soon as they can. Most of the rest have strong local ties or have just given up and collect their pay.

THE LEVELS OF SOFTWARE PROCESS MATURITY

Level 1—Initial
Characteristics: Chaotic—unpredictable cost, schedule, and quality performance
Needed Actions: Planning (size and cost estimates and schedules), performance tracking, change control, commitment management, Quality Assurance

Level 2—Repeatable
Characteristics: Intuitive—cost and quality highly variable, reasonable control of schedules, informal and ad hoc process methods and procedures
Needed Actions: Develop process standards and definitions, assign process resources, establish methods (requirements, design, inspection, and test)

Level 3—Defined
Characteristics: Qualitative—reliable costs and schedules, improving but unpredictable quality performance
Needed Actions: Establish process measurements and quantitative quality goals, plans, measurements, and tracking

Level 4—Managed
Characteristics: Quantitative—reasonable statistical control over product quality
Needed Actions: Quantitative productivity plans and tracking, instrumented process environment, economically justified technology investments

Level 5—Optimizing
Characteristics: Quantitative basis for continued capital investment in process automation and improvement
Needed Actions: Continued emphasis on process measurement and process methods for error prevention

.2 A Case Study of a Chaotic Project

I once participated in the review of a large military software development contract. The project was seriously late, and the review was to see if the current schedule would be met. After extended discussion we concluded the basic system should be completed within a few months of the current schedule. We also learned that a small amount of additional function was required but that it would not present a problem. We did not, however, see a plan. We reported that the project was in good technical shape but that there was modest further schedule exposure. While we were concerned about the lack of documented planning, the technical work looked good, and we really did expect the project to finish nearly on schedule.

After this report I periodically checked to see if the project had been completed. Every time, I was told it was almost done. Some months later I heard that the "small amount" of added code turned out to be 250,000 lines! It had seemed so simple that no one bothered to make a plan. Without a plan, they had no idea how big the job really was, no basis to know where they stood, and no way to justify added resources when they got into trouble. Even though the technical work was sound, the customer was convinced the project would be a disaster. The project was ultimately completed and was reasonably successful, but the customer lost so much confidence in the software people that he refused to pay for much of this work. As is often the case with the Initial Process, inadequate project management nearly destroyed years of good technical work.

3. Why software organizations are chaotic

The simplest explanation for the chaotic behavior of Level 1 organizations is human nature: People don't like to admit they don't know. Unfortunately, in software development there are an enormous number of unknowns. Without some work we generally have no idea how much code a given function will require or how much effort it will take. What is worse, in many cases the software team doesn't even know how to go about finding out.

When political pressures mount for a commitment, it takes guts to say you don't know. This is known as betting your job. If you insist on taking the time to make a good plan, you may be replaced by someone who promises to be more responsive. On the other hand, if you cave in to the pressure, you have to come up with a number. With nothing to guide you, you play it safe and put in the largest number the traffic will bear.

While lack of a commitment discipline is the most common reason for chaotic behavior, there are other forces.

1. Under extreme pressure, s/w managers often make a guess rather than a plan. When the guess is drastically low, chaos ensues. Intuitive commitments occasionally are accurate, but generally only when the project scale and function are similar to the guesser's prior experience.
2. When the going gets rough, there is a strong temptation to believe in magic. Some savior may appear, or a new technology may be the answer. Since this hope is often an excuse for not planning, it merely postpones the problem.
3. The scale of s/w projects follows an escalating cycle:
    - Programs generally take far more code than expected.
    - As the programs become larger, new technical and management issues come up.
    - Since these are unlike previous experience, they are a surprise.
    - As the scale increases, the surprises continue, but at dramatically increased cost.

1. Chaotic Forces-Unplanned Commitments

The most difficult s/w management problem is the unplanned commitment.

Minor Commitments: When a new requirement seems simple, the manager is tempted to merely commit to do it. Since the s/w is a matter of detail, simple looking functions often have hidden traps. When these are not found until after commitment, the manager is in an impossible situation. The normal result is an inadequately staffed crash effort to meet the date, regardless of job size.

The unplanned commitment trap: The unplanned commitment is a trap because of the forces it generates. First, lack of an orderly plan generally means there are insufficient time and resources to do the job. This results in confusion, no time to think through, and an unwillingness to use any but the lowest risk, brute force methods. Under these conditions things get worse.

2. Chaotic Forces-Gurus and magic

The technical wizard can be a powerful asset. Unfortunately, gurus sometimes believe they can do no wrong. After they have led a small team, they believe they can do anything. During a time of crisis there is no way to recover as most gurus run project out of their heads.

While there are many cases in which improved technology can help, there are many more that need effective management. It is both understandable and desirable to search for better methods and people.

3. Chaotic Forces-Problems of scale

One of the fundamental reasons for problems of scale is the large size of many programs.

As s/w becomes larger, they are much more difficult to understand.

1. One person knows the details of the program
2. One person understands it but can't remember it so the design must be documented.
3. One person understands the overall design of a program, but the details of its component modules are each understood by separate experts
4. A large s/w product is defined and understood at the product management level, but a separate team understands the characteristics and design of each of its component programs.
5. With s/w systems, the high level design may be well defined and understood by the system management team, but each of its component product is only understood by the respective product management organizations.
6. When the system is very large and has evolved through many versions, there may be no one who understands it.

As s/w knowledge is more widely distributed:

1. Common notations are needed for precise communication.
2. These standards must be documented, interpreted and updated
3. Conflicts in standards must be identified and resolved
4. Standard changed must be controlled and distributed

As s/w size increases, prototypes or multiple releases are needed because:

1. The total function cannot be implemented in time.
2. Some need cannot be understood without operational experience on a partial system
3. Some design issues cannot be resolved until a preliminary system has been built and run
4. A release discipline helps sort out user priorities
5. No one successfully builds large s/w systems in one shot way

5. Implication of software Scale

The problems of software scale affect the individual, the management system, and the technical methods and tools that we use. Most people begin their software careers by writing small programs. They find it so stimulating and rewarding to produce a program that runs that they think they have the software problem licked. They "know" how to write programs and wonder what all the fuss is about.

4. Software Process Entropy:

There are many forces on the s/w process that push us toward disorganization.

1. Dynamic Requirements

2. Increasing system size

3. Human nature are the three classes of forces that tend to disrupt it.

The evolutionary process is driven by requirement dynamics. Each new development uncovers new issues that lead to changes. Changes are highly error prone and thus disruptive. Change disruption is thus compounded by increasing project scale.

In large s/w systems, he most severe problems are not obvious until testing gets into trouble. Without a plan, the response driven management system treats testing as a problem. Such irrational priorities produce increasingly chaotic behavior.

5. The way out:

The general solution for chaos of the initial process is:

1. Apply systematic project management-the work must be estimated, planned and managed.
2. Adhere to careful change management-changes must be controlled, including requirements, design, implementation and test.
3. Utilize independent s/w assurance-an independent technical means is required to assure that all essential project activities are properly performed.

The basic principles for controlling the chaos in software organizations are:

1. Plan the work
2. Track and maintain the plan
3. Divide the work into independent parts.
4. Precisely define the requirements for each part
5. Rigorously control the relationships among the parts
6. Treat s/w development as a learning process
7. Recognize what you don't know
8. When the gap between your knowledge and the task is severe, fix it before proceeding.
9. Manage, audit, and review the work to ensure is done as planned
10. Commit to your work and work to meet your commitments.
11. Refine the plan as your knowledge of the job improves.

5. The Repeatable Process

Repeatable Process includes Commitments, Planning, configuration management and Quality Assurance.

Managing Software Organizations

The role of the management system is to ensure that projects are successfully completed.

The basic principles of project management are:

- Each project has a plan that is based on a hierarchy of commitments.
- A management system resolves the natural conflicts between the projects and between the line and staff organizations.
- An oversight and review system audits and tracks progress against the plans.

Making Commitment:

Commitment is an agreement by one person to do something for another. Typically this involves a planned completion date and some consideration or payment.

For eg::two programmers cooperate on a single project, they must divide up the work. This division starts with agreement on the distribution of functions, common interfaces, standard formats, and naming conventions. For either programmer to perform effectively, each must know what the other is doing and be able to rely on its being done is agreed. This reliance is achieved through commitment process.

The elements of effective commitments are:

1. The person making the commitment does so willingly
2. The commitment is not made lightly; that is, the work involved, the resources, and the schedule are carefully considered.
3. This agreement between the parties on what on what is to be done, by whom and when
4. The commitment is openly and publicly stated
5. The person responsible tries to meet the commitment, even If help is needed.
6. Prior to committed date, if it is clear that it cannot be met, advance notice is given and a new commitment is negotiated.

The software commitment Process:

To be effective, the software commitment process must reach to the top of the organization.

This requires:

1. All commitments for future s/w delivery are personally made by the organizations senior executive.
2. These commitments are made only after successful completion of a formal review and concurrence process.

3. An enforcement mechanism ensures that these reviews and concurrences are properly conducted.

To do this, the senior manager should require evidence that the following work has done prior to approving a commitment:

1. The work has been defined and agreed to between the developers and their customer.
2. A documented project plan has been produced, including a resource estimate, aschedule, and a cost estimate.
3. All directly involved parties have agreed to this plan in writing.
4. Adequate business and technical preparation has been done to ensure that the commitment represents a reasonable risk.
5. An independent review has been conducted to ensure that this planning work was done according to the organizations standards and procedures.
6. The groups participating in the work have or can acquire the resources required.

The Management System

While every organization will have unique objectives, there are generally four components:

1. To have a technical and business strategy that aims at such long term goals as growth rate or market position
2. To provide quality products that meet customers needs in a timely and effective way
3. To perform the assigned mission competitively and economically
4. To perform continually the organizations ability to handle more challenging work

These general objectives, applicable to most organizations, demonstrate the inherent conflicts common to most software groups.

Project Plan

Project Planning Principles

The project plan is developed at the beginning of the job and is successively refined as the work progresses. The logic for s/w project planning is:

1. While requirements are initially vague and incomplete, a quality program can only be built from an accurate and precise understanding of users needs

2. A conceptual design is then developed as a basis for planning

3. At each subsequent requirements refinement, resource projections, size estimates and schedules are also refined.

4. When the requirements are clear, a detailed design and implementation strategy is developed and incorporated in the plan

5. Implementation details are established and documented in further plan refinements

6. The plan provides the framework for negotiating the time and resources to do the job.

The planning cycle:

1. The cycle starts with the initial requirements
2. The response to every demand for a commitment must be to produce a plan with objectives
3. The plan is produced by first breaking the work into key elements, called a work breakdown structure(WBS)
4. The size of each product element is estimated
5. The resource needs are projected
6. The schedule is produced

Project Plan Contents:

The elements of a s/w plan are:

- Goals and objectives: these describe what is to be done, for whom and by when
- Work Breakdown Structure (WBS)The WBS subdivides the project into tasks that are each defined, estimated and tracked.
- Product Size Estimates: these are quantitative assessments of the code required for each product element.
- Resource estimates: Based on prior experience, known productivity factors are applied to yield reasonable estimates of the resources required for each WBS element
- Project Schedule: Based on the available project staffing and resource estimates, a schedule for the key tasks and deliverable items is produced.

6. The Defined Process

Software Standards

A standard is a rule or basis for comparison that is used to assess the size, content, value or quality of an object or activity.

Definitions:

- *Policy*: A governing principle, typically used as a basis for regulations, procedures, or standards and generally stated by the highest authority in the organization.
- *Regulation:* A rule, law, or instruction, typically established by some legislative or regulatory body with penalties for non-compliance
- *Specifications:* The precise and verifiable description of the characteristics of a product.
- *Guideline*: A suggested practice, method or procedure, typically issued by some authority.
- *Procedure:* A defined way to do something generally embodied in a procedures manual.
- *Standard:* A rule or basis for comparison that is used to assess size, content or value, typically established by common practice or by a designated standards body

Standards are needed when many people, products or tools must coexist. They are essential for establishing common support environments, performing integration, or conducting system test.

Standards also promote the consistent use of better tools and methods.

Establishing Software Standards

The establishment of standards starts with an examination of the organizations standards and procedures needs. These should be considered in three categories:

1. Management and planning standards and procedures
   - Configuration management
   - Estimating and costing
   - Software Quality Assurance
   - Status Reporting
2. Development process standards and methods
   - Requirement specification
   - Design
   - Documentation
   - Coding
   - Integration and testing
   - Reviews, walkthrough and inspections
3. Tool and process standards
   - Product naming
   - Size and cost measures
   - Defect counting and recording
   - Code entry and editing tools
   - Documentation systems
   - Languages
   - Library system

The standards Development Process

Standards development involves the following steps:

1. Establishing a standards strategy that defines priorities and recognizes prior work
2. Distribute, review, and maintain this strategy
3. Select individuals or small working groups to develop the top priority standards
4. This development effort should build on prior work where available, define the areas of applicability, specify the introduction strategy, and propose an enforcement plan
5. The draft standards should be widely distributed and reviewed.
6. The standards should be revised to incorporate the review comments and the re reviewed if the changes are extensive
7. The standards should initially be implemented in a limited test environment
8. Based on this test experience, the standards should again be reviewed and revised
9. Implement and enforce the standards across the defined areas of applicability

10. Evaluate the effectiveness of the standards in actual practice.

The work of establishing a standard should be done by individuals or small working groups of technical experts. In establishing a new standard, it is rarely wise to adopt one from another organization without carefully examining its fit to the unique needs of the local users. Since standard reviews serve both a management and a technical purpose, they should involve all facets of the organization.

Maintaining Standards:

Standards must be kept current. They should be modified and adjusted based on the experiences in using and enforcing them, on the changes in available technology, and on the varying need of the projects.

The key responsibilities of standard maintenance are:

1. Stay aware of standards implementation problems.
2. Be a source of guidance and advice on using the standard
3. Maintain the standards baseline and control changes
4. Periodically review the standards to ensure their effectiveness and pertinence.

Standards maintenance should not involve a great deal of work. If a standard need frequent changes, it probably covers a subject that is not ready for standardization.

The standards and procedures should also be reviewed at least once every three to five years to ensure they are current and needed.

Software Inspection

Software inspections provide a powerful way to improve the quality and productivity of the software process. The fundamental objective of inspections is to improve the quality of programs by assisting programmers to recognize and fix their own errors early in the software engineering process. While inspection will not solve all problems, they are enormously effective.

Inspection Objectives:

The basic objectives of inspections are:

1. To find errors at the earliest possible point in the development cycle
2. To ensure that the appropriate parties technically agree on the work
3. To verify that the work meets predefined criteria
4. To formally complete a technical task
5. To provide data on the product and the inspection process.

Inspection also provides a host of secondary benefits:

1. They ensure that associated workers are technically aware of the product
2. They help to build an effective technical team
3. They help to utilize the best talents in the organization
4. They provide people with a sense of achievement and participation

5. They help the participants develop their skills as reviewers.

Basic Inspection Principles:

The inspection process follows certain basic principles:

1. The inspection is a formal, structured process with a system of checklists and defined roles for the participants. It provides an orderly means to implement a standard of software engineering excellence throughout an organization.
2. Generic checklists and standards are developed for each inspection type and where appropriate, they are tailored to specific project needs.
3. The reviewers are prepared in advance and have identified their concerns and questions before the inspection starts.
4. The focus of the inspection is on identifying problems, not resolving them
5. An inspection is conducted by technical people for technical people.

The conduct of inspections

Inspections should be conducted at every point in the development or maintenance process at which intermediate products are produced.

The inspection participants include the following people:

1. The moderator(or inspection leader) The person responsible for leading the inspection expeditiously and efficiently to a successful conclusion
2. The producers The person or persons responsible for doing the work being inspected
3. The reviewers(or inspectors) Generally people who are directly concerned and aware of the work being inspected
4. The recorder(or scribe) someone who records the significant inspection results.

Preparing for the inspection

The full inspection process consists of a preparation phase, the inspection itself and post inspection activity. As a first step the producers and their manager decide that the product is ready for inspection and agree on the inspection objectives. Next the inspection participants are identified, the inspection entry criteria are prepared, and the supporting materials are produced for the opening meeting with the entire inspection group.

In conducting the inspection meeting, the moderator first checks to see if all participants are prepared and obtains copies of any preparation reports not already submitted. The producer then review each major error. Following the inspection, the producer fixes the identified problems and either reviews the corrections with the moderator or in reinsertion

Software Testing

Software Testing is defined as the execution of a program to find its faults.

Testing: The process of executing a program with the intentions of finding errors

Verification: An attempt to find errors by executing a program in a test or simulated environment

Validation: An attempt to find errors by executing a program in a real environment

Debugging: Diagnosing the precise nature of a known error and then correcting it.

The 7 types of Software Tests:

1. Unit or module tests verify single programs or modules. These are typically conducted in isolated or special test environments
2. Integration tests verify the interfaces between system parts
3. External function tests verify the external system functions, as stated in the external specifications
4. Regression tests run a subset of previously executed integration and function tests to ensure that program changes have not degraded the system
5. System tests verify and/or validate the system to its initial objectives
6. Acceptance tests validate the system or program to the users requirements
7. Installation tests validate the installability and operability of the users system.

Testing Methods:

There are 2 basic ways of constructing tests: white box and black box. White box tests examine the internal design of the program and require that the tester has detailed knowledge of its structure. Black box tests, however are designed without knowledge of the programs internals and are generally based on the functional requirements

Axioms of Testing:

Test hypotheses concern the types and quantity of defects in the program. The testing axioms stated by Myers are:

1. A good test case is one that has a high probability of detecting a previously undiscovered defect
2. Difficulty in testing is knowing when to stop
3. It is impossible to test your own program
4. A necessary part of every test case is a description of the expected output
5. Avoid non reproduceable testing
6. Thoroughly inspect the results
7. Ensure that testability is a key objective in your software design
8. The design of a system should be such that each module is integrated into the system only once
9. Never alter the program to make testing easier
10. Testing, must start with objectives

Test Planning

Major Test plan elements are:

1. Establish objectives for each test phase
2. Establish schedules and responsibilities for each test activity
3. Determine the availability of the tools, facilities and test libraries
4. Establish the procedures and standards to be used for planning and conducting the tests and reporting the test results
5. Set the criteria for test completion as well as for the success of each test.

Real time testing

1. Devise ways to exercise each system state either in real time or in simulated parametric time
2. Design ways to simulate any modules missing in the current test phase so the real time tests can operate
3. Evaluate possible deadlocks, thrashing, or sensitivity to special timing conditions
4. Use tests that simulate hardware faults or special system conditions
5. Use hardware simulation to stress the s/w design

Defining the software process

Process Standards

1. Process standardization helps to reduce the problem of training, review and tool support
2. Each projects experience can contribute to overall improvement
3. Process standards provide the basis for process and quality measurement
4. Since process definitions take time and effort to produce, it is impractical to produce new ones for each project

Software engineering: the disciplined application of engineering, scientific and mathematical principles, methods and tools to the economical production of quality s/w

s/w engineering process: The total set of s/w engineering activities needed to transform a users requirement into s/w

s/w process architecture: A framework within which project specific s/w process are defined

s/w process model: One specific embodiment of a s/w process architecture

S/w process the set of activities, methods and practices that are used in the production and evolution of software

Levels of s/w process models

Three levels: U or Universal process model, W or worldly process model and A or Atomic Process model

U Process Model

The waterfall model is widely used s/w.

It has several shortcomings:

1. It does not adequately address changes
2. It assumes a relatively uniform and orderly sequence of development steps
3. It does not provide for such methods as rapid prototyping or advanced languages

To address these concerns, Bohem proposed Spiral Model. This U level model provides a high level overview of some of the issues.

A Process Models

Atomic level process models are enormously detailed. Precise data definitions, algorithmic specifications, information flows and user procedures are essential at this level. Atomic process definitions are often embodied in process standards and conventions.

W Process Models

The Worldly process level is of most direct interest to practicing s/w engineers. It guides the sequence of their working tasks and defines task prerequisites and results. When reduced to operational form, these models look like procedures.

Critical S/w process Issues

The reason for defining the software process is to improve the way the work is done.

Some common issues are:

- *Quality:* humans are error prone. Each error when found is a surprise whose correction is both expensive and disruptive.
- *Product Technology:* Often with new developments it is not clear how to implement an algorithm, to meet performance goal or to pack function into a limited configuration. It is wise to make process provisions as patching of unsuccessful attempts can be time consuming and disruptive. Subsequent development can then proceed in a more orderly and effective way.
- *Requirements Instability:* to design, build and to test a program the required functions, interfaces and environments must be stable. While these may change during development, the changes must be temporarily frozen while development proceeds. There are three basic types of requirement changes
    i. Unknown requirements
    ii. Unstable requirements
    iii. Misunderstood requirements
- *Complexity:* Application programs are often easier to develop than complete systems because their environment is generally more stable.

The Software Engineering Process Group

Software process improvement is not simple. To achieve tangible results, it needs to be treated like a development task. The SEPG has 2 basic tasks that are done simultaneously: initiating and sustaining process change and supporting normal operations. Process change requires the

professionals to learn new methods and to accommodate to the changing nature and scale of the problems they encounter.

SEPG Sustaining Role

The role of the SEPG can be divided into 6 categories:

1. Establish process standards
2. Maintain the process database
3. Serve as the focal point for technology insertion
4. Provide key process education
5. Provide project consultation
6. Make periodic assessments and status reports.

7. The Managed Process

Data Gathering and Analysis

The Principles of Data Gathering:

1. The data gathered in accordance with specific objectives and a plan
2. The choice of data to be gathered is based on a model or hypothesis about the process being examined
3. The data gathering process must consider the impact of data gathering on the entire organization
4. The data gathering plan must have management support.

The objectives of Data Gathering:

There are four basic reasons for collecting software data:

1. Understanding As part of a research or development study,data can be gathered to learn about some item or process
2. Evaluation The data can be used to study some product to see if it meets acceptance criteria
3. Control The data can be used to control some activity
4. Prediction The data can be used to develop rate or trend indicators

The Data Gathering Process

1. Managing the data Gathering Process

Data gathering is expensive so its process must be carefully planned and managed. The data must be precisely defined to ensure that the right information is obtained and it must be validated to ensure that it accurately represents the process.

2.Data Gathering Plan

The data gathering plan should be produced by SEPG. It should cover the following topics:

a) What data is needed, by whom, and for what purpose? The plan must clearly and precisely state how the data is to be used.

b) What are the data specifications? These include the definition of the initial data to be gathered and the plan for defining any additional data that will likely be needed.

c) Who will gather the data? The people who will gather the data are identified

d) Who will support data gathering? A training program is provided, and a support group is available to answer questions

e) How will the data be gathered? Appropriate forms and data entry facilities are made available, and suitable procedures are established

f) How will the data be validated? Since data is error prone it should be validated as rapidly as practical

g) How will the data be managed? One can gather enormous amounts of data on software projects.

3. Data Validation

The way data is validated depends on the data involved. With time measures, the total elapsed time can be compared to the total of the elements.

S/w Measures:

Data characteristics:

S/W process measures should have the following characteristics:

- The measures should be robust
- The measures should suggest a norm
- The measures should relate to specific product or process properties
- The measures should suggest improvement strategy
- They should be a natural result of the process
- The measure should be simple
- They should be both predictable and traceable

S/W measurements can be classified in several ways:

- Objective/subjective: This distinguishes between measures that count things and those involving human judgment
- Absolute/relative: Absolute measures are typically invariant to the addition of new items. The size of one program is an absolute measure and is independent of the sizes of the others. Relative measures change, as with an average or a grading curve.
- Explicit/derived: Explicit measures are taken directly, while derived measures are computed from other explicit or derived measures.

- Dynamic/static: Dynamic measures have a time dimension, as with errors found per month. Static measures remain invariant, as with total effort expended or total defects found during development.
- Predictive/explanatory: Predictive measures can be obtained or generated in advance, while explanatory measures are produced after the fact.

Definitions

Errors: These are human mistakes and their quantification often depends on an understanding of the programmer's intentions

Defects: These are improper program conditions that are generally the result of an error.

Bugs: A bug or fault is a program defect that is encountered in operation, either under test or in use.

Failures: A malfunction of user's installation

Problems: User encountered difficulties.

Classes of Defect Measures:

Software defects can be categorized as:

1. Severity: Measures the actual or anticipated impact of a defect on the users operational environment
2. Symptoms: refers to observed system behavior when the defect is found
3. Where found: by the system location where they were identified
4. When found: when they are found in the s/w life cycle, as in unit test, function test, system test etc.
5. How found: operations being performed when the defect was found.
6. Where caused
7. When caused: identifies defects as introduced during high level design
8. How caused: counts of the errors that caused the defects can be most useful in improving the s/w process.
9. Where fixed: record where changes are made when installing fixes.
10. When fixed: helpful in evaluating and improving the maintenance and enhancement process.
11. How fixed: how change is designed and applied

Managing Software Quality

One of the best ways to evaluate a s/w organization is to examine the quality of its products.

Classes of Quality Measures:

Quality Measures fall into following general classes:

1. Development : Defects found, change activity
2. Product: Defects found, s/w structure, information structure, controlled tests

3. Acceptance: Problems, effort to install, effort to use
4. Usage: problems, availability, effort to install, effort to use, user opinions
5. Repair: Defects, resources expended

These measures are characterized as follows:

1. Objective: can the measure be repeatedly produced by different people
2. Timely: Is it available in time to affect the development or maintenance process?
3. Available: how hard is it to obtain?
4. Representative: To what extent can it represent customer's view of goodness?
5. Controllable: To what extent can its value be changed by development or maintenance actions?

Making a S/W quality Estimate

1. Recent programs completed by the organization are reviewed to identify the ones most similar to the proposed product.
2. Available quality data on these programs is examined to establish a basis for the quality estimate
3. The significant product and process differences are then determined and their potential effects estimated.
4. Based on these historical data and the planned process changes, a projection is made of anticipated quality for the new product development process
5. This projection is then compared with goals and needed process improvements are devised to meet the goals
6. The project quality profile is then examined to determine the areas for potential improvement and a desired quality profile is produced
7. A development plan is produced that specifies the process to be used to achieve this quality profile

Tracking and Controlling S/W Quality:

The critical elements of a s/w quality management system are:

1. A responsible authority is named to own the quality data and the tracking and reporting system
2. Quality performance is tracked and reported to this authority, during both development and maintenance
3. Resources are established for validating the reported data and retaining it in the process database.
4. Actual product and organizational performance data is periodically reviewed against plan:
    - Results are initially reviewed with responsible line management and any discrepancies involved
    - Performance against targets is determined, and, if worse than plan, line management prepares an action plan for review with higher management.

- If actual performance is substantially better than plan, product management establishes more aggressive targets.

5. Quality performance is published, and a highlight report is provided to senior management
6. Overall performance is periodically reviewed with senior management.

8. The Optimizing Process

Defect prevention

The principles of s/w defect prevention:

The fundamental objective of s/w defect prevention is to make sure that errors, once identified and addressed, do not occur again.

The principles are:

1. The programmers must evaluate their own errors.
2. Feedback is an essential part of defect prevention
3. There is no single cure-all that will solve all the problems
4. Process improvement must be an integral part of the process
5. Process improvement takes time to learn

The steps of s/w defect prevention

1. Defect Reporting: This includes sufficient information to categorize each defect and determine its cause
2. Cause Analysis: the cause of the most prevalent defects are determined
3. Action plan development: Action teams are established to devise prevention for the most prevalent problems
4. Action implementation: Once the actions have been determined, they must be implemented.
5. Performance tracking: Performance data is gathered, and all action items are tracked to completion
6. Starting over: Do it all again, only this time focus on the most prevalent of the remaining defects.

Automating the s/w process

The reason for automating the s/w process is to improve the quality and productivity of our work. Organizations need an automation strategy to guide their use of new methods and techniques. This requires knowledge of what is needed, awareness of what is feasible and development of an orderly plan.

The long term automation plan:

1. Establish an automation focal point: this involves naming an individual to lead the study and planning efforts.

2. Next, define current automation status, establish immediate priority need, and gather information on pertinent available tools and methods.
3. An orderly assessment is next made of the most promising available environments and tools
4. Establish a strategic plan for environmental support
5. Define a migration plan to take the organization from its current support systems to the strategic environment.
6. It is needed to provide a clear framework for selecting suitable interim tool sets and migration plans.

Implementing an Automation Plan

1. An implementation support group is established to handle the ordering, vendor negotiations, installation, conversion, operation and support for the environment.
2. A more detailed automation plan is developed.
3. Nest, specific project automation objectives and plans are developed, consistent with this overall plan.
4. Each project then develops a specific migration plan with agreed responsibilities and milestones for each key transition step.
5. The new environment will require the programmers to change the ways they work.

Contracting for software

Software Contracting:

It is instructive to view the s/w process as a contract between management and the development organization. The principles of s/w contract management are essentially the same regardless of the buyers technical competence. The basic system objectives are established at the outset, where specific functional requirements are known. A documented plan is then produced for establishing and validating the requirements and the operational concept against these objectives, and a joint requirements effort defines the system tasks with sufficient precision to permit design to start.

Software contracting conditions:

1. Start with a very precise statement of what was wanted
2. Insist on rigid standards and detailed documentation of every process step
3. Require that each step be completed and approved before the next was initiated
4. Demand a firm commitment at the outset

The principles of effective software contract management

The key criteria for successful software contracting are:

1. The vendor must be trustworthy and technically competent
2. The buyer must be capable of identifying technically competent vendors
3. The contract presumes mutual trust
4. SQA and audit ensure honest and disciplined performance

5. Initially the highest priority task is to arrive at agreement on what is to be produced, a plan to produce it, and acceptance criteria
6. It is recognized that the operational concept and the product requirements will evolve throughout the contract and that appropriate provisions will be made to handle the resulting scope and plan changes.

Management must develop a system for tracking projects and commitments.

1.7 The Defined Process

Why define a process?

There are a lot of reasons why an organization might decide that it's time to define a process. Usually workers and management start noticing that there is a lot of time being spent not on the actual work, but on fire-fighting (e.g., doing rework, fixing, re-fixing, testing, retesting, putting out fires caused by tasks that weren't done at all, repairing relationships with customers, and running around to see if anyone remembers what worked last time). We've all seen ten people doing the "same" work without a process—no one agrees on exactly what is done, or how it should be done, when, or by whom. That the work gets done at all may be surprising. When an organization goes into fire fighter, or reactive mode, the solution often seems to be to hire more people. So now we can have twelve or fourteen people putting out fires, instead of just ten.

That's sure to improve productivity and quality. Eventually, someone may decide that it's time to turn off the sirens, do some analysis, put together a team, and get the process defined, implemented, and managed.

Some benefits of process work include

1. Increase the effectiveness of preventive measures,
2. Reduce variation, increase reliability,
3. Reduce redundant or non-critical work,
4. Reduce rework and retesting,
5. Increase the cooperation between departments or teams,
6. Improve cycle-time and quality, and
7. Improve the ability to forecast and estimate.

Process definition and management efforts ultimately affect customer satisfaction, employee morale, and the organization's profitability.

Defining A Process: Overview

The following diagram illustrates the steps to follow in defining a process. Three roles are involved: the Sponsor, the Facilitator, and the Process Definition Team. The Sponsor the Sponsor enables and motivates the Process Definition Team by mapping the organization's vision, mission, and principles to the process team's scope and goals. The Sponsor demonstrates to the team that the work they are doing is a priority by helping to remove barriers and ensuring that the team has the resources they need. The best candidate for a Sponsor is a high-level manager who is responsible for at least part of the functions in the process and can be involved with the Process Definition Team.

The Facilitator

The Facilitator leads the team through the definition, implementation, and management of the process. The Facilitator needs to have skills in group facilitation, team building, metrics and measurement theory and practice, and problem-solving tools, as well as practical experience in process definition.

The Process Definition Team

The Process Definition Team is comprised of workers from the organizations that are most involved in the process being defined. Teams must include workers from the level "where the action is," that is, the workers who actually perform the steps in the process. The team should not have more than six primary members. If there are more than six departments or functions involved in the process, workers from groups with less involvement in the process can meet with the primary team as needed.

Reading the Process Diagram

The conventions used in the diagram "How to Define a Process" were designed to make the process flow diagram easy to read for technical and non-technical staff alike. The diagram clearly shows who is doing what, and when.
The following conventions were followed for this diagram, and are recommended in defining all processes:
1. Each role in the process has a horizontal "action zone" showing steps performed by persons in that role.
2. Steps that overlap action zones are steps performed together.
3. Steps with dotted-line borders are "ongoing" steps that continue to be done throughout the process, as needed.
4. A step with heavy, thick borders is an "external process" step, indicating a separate process being called.

The Steps In Defining A Process

The following is a condensed version of the step descriptions for each step in the process flow diagram. Information on obtaining the complete documentation follows of this paper.

Step 1—Initiate Process Definition

The Sponsor and the Facilitator work together to provide the framework for the Process Definition Team. It is the responsibility of the Facilitator to explain the benefits of process definition and implementation, how the work will be accomplished, and how much effort will be involved. The Facilitator explains to the Sponsor about the importance of having team members who actually do the work, and asks the Sponsor for a list of potential team members. The Sponsor provides information about the organization and the process to be defined.

The purpose of this step is to see that the following are fully documented:

1. Organization Vision and Mission Statement
2. Process description, and description of how the process helps with the organization's mission
3. Scope of the process. When does the process start? End? What activities, functions, or outputs are not to be included in this process? Which organizations are involved in this process?

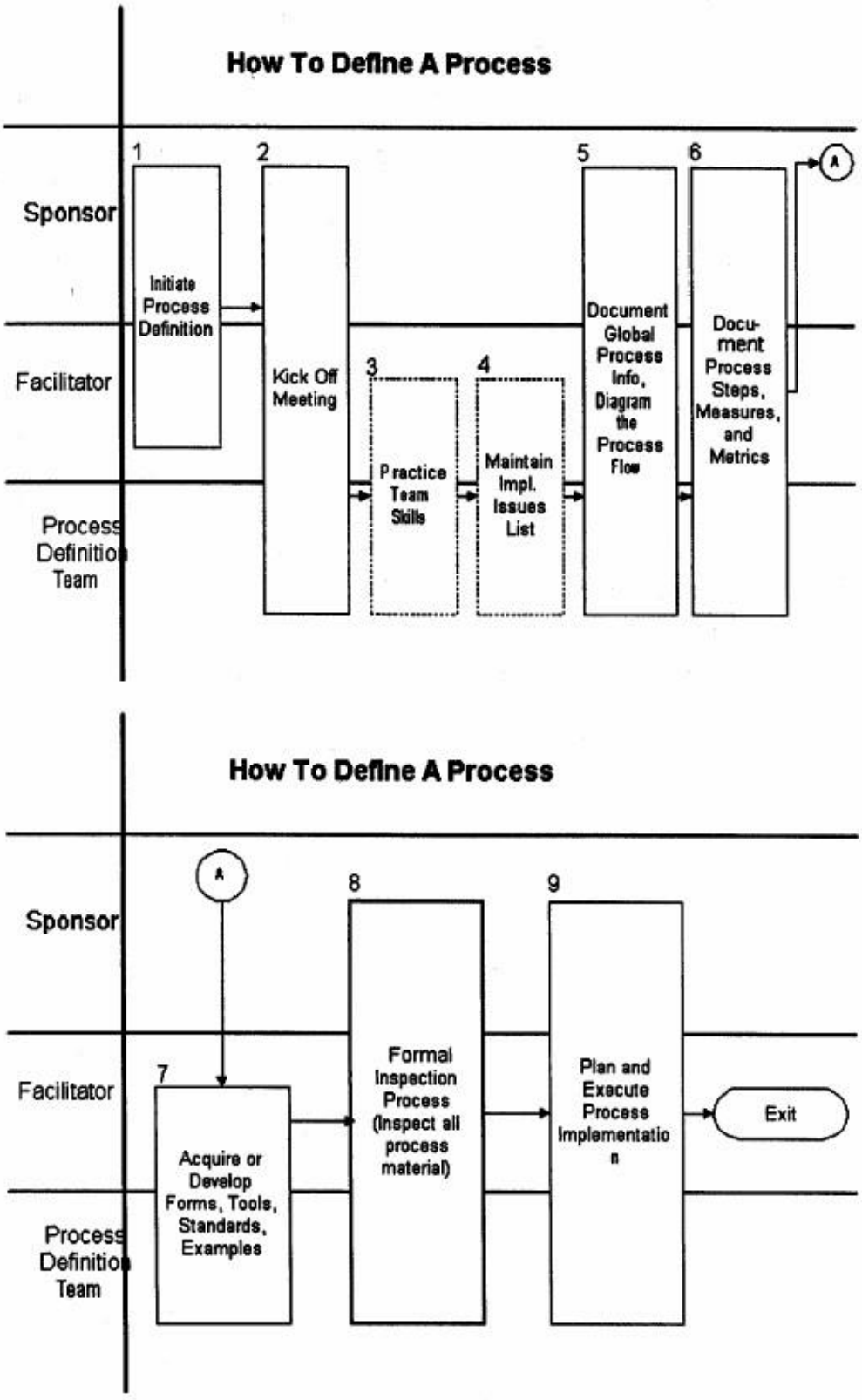


Figure 2: How to Define a Process

4. Current state of the process (e.g., what is working? What is not? Is there consistency?
   Redundancy? Extra Work?)
5. Process goals (What should this process accomplish? How will you know the process is a success? What should be measured? What will indicate a problem

with the process?)
6. Specific improvements to be addressed
7. Roles involved in the process
8. Primary internal and external customers of the process
9. Boundaries and constraints for the team (e.g., time allotted, target dates, equipment, costs, personnel, etc.)
10. Potential Process Definition Team members.

Step 2—Kick Off Meeting

The new Process Definition Team meets with the Sponsor and Facilitator to kick off the process work. The Sponsor and Facilitator have contacted people from the list of potential team members and now have a list of the team members. They now plan a meeting, provide the agenda, schedule the meeting, and lead the meeting.

The Kick Off Meeting is designed to:

1. Formalize involvement in the Process Definition Team,
2. Discuss the documented process goals and scope and verify that everyone is working with the same definitions,
3. Provide team members an overview of the process definition work, and,
4. Start the team working together right away by defining team rules and setting a meeting schedule.

Suggested Kick Off Meeting Agenda Items

- Process Sponsor - Introductions, purpose of meeting
- Process Sponsor - Presentation of process goals, scope
- Question and answer period
- Facilitator - Overview of process definition tasks
- Facilitator - Action Item assignment: who will handle the Action Item Log for the team? The Parking Lot log?[1]
- Facilitator - How often will the team meet? When? Where? Who will schedule the meetings and book the conference room?
- Facilitator - Work with team to decide on team name and team meeting rules
- Sponsor - Closing remarks

Step 3—Practice Team Skills

The Process Definition Team will be learning how to work together as a team in which each member contributes, and in which tools are used to make decisions and solve problems. The Facilitator needs to watch for opportunities to apply team-building exercises, problem-solving tools, and decision-making techniques. Tools that will be most used in process definition include brainstorming, negative brainstorming, affinity diagramming, selection matrix, cause/effect analysis, nominal group technique, data collection, Pareto Charts, force field analysis, plan-do- check-act cycle, and multivoting.

Suggested Reference Material for the Facilitator

The Team Handbook (How to Use Teams to Improve Quality) by Peter R. Scholtes and *The Quality Toolbox* by Nancy R. Tague. Both are available from ASQ Quality Press.

Step 4—Maintain Implementation Issues List

During the definition of the process, implementation issues will be identified. The planning for implementation starts right away, with the creation of an Implementation Issues List. Topics are grouped and prioritized. Typical implementation issue types include training, tools, forms and templates. Implementation planning is an ongoing step until the process is completely documented. At that point, the Implementation Plan is formalized by complete documentation and inspection.

The Facilitator should be sure that the Process Definition Team identifies human factors that will affect implementation of the process. The team must plan ways to get non-team members involved with the process definition from the start; monthly presentations of the team's progress to their co-workers may be a good way to do this.

The implementation plan will have the most chance of success if the following are considered:

1. Everyone in the involved departments should understand that there must be a change to the way work is done, and why.
2. Everyone should understand the long term and short term goals, and how that will affect their department, their team, themselves.
3. Everyone should understand the steps that will be taken to meet those goals.
4. Most co-workers trust the Process Definition Team to be sure that before a change in work procedure is made, there will be training, tools, and examples provided.
5. Some people may require more coaching than others in new procedures. Everyone must understand that the Process Definition Team members will do everything they can to be sure that each individual can succeed in following the process to do their work.
6. It must be clear to everyone that the measurements and metrics defined for this process will never be used against an individual, but will be used for process improvement purposes.

Step 5—Document Global Process Information and Diagram the Process Flow

The Global Process Information

The Global Process Information is basically taken from Step 1—Initiate Process Definition, when the Sponsor provided information about the process scope and goals. At this point, the Process Definition Team will assign one individual member to be responsible for completing this documentation. The team can use the draft as a working copy, improving it until the inspection in Step 8.

The Contents of the Global Process Information Section

The Global Process Information is the first section of the Process Definition Document. It contains the following sections:

1. Overview (Basic description of the process.)
2. Objectives of the Process
3. Process Scope/Start and End
4. Parties Involved in the Process
5. Process Owner
6. Tools, Standards, Forms Used by this Process (A table listing name, use, and locations.)
7. Terms and Definitions
8. Related Documents (References to related documents, includes document

name, description, and location. Include reference to the Process Metrics Plan.)

The Process Flow Diagram

The process flow diagramming is done in one or more team meetings, with the Facilitator leading. The recommended diagram type is that used in this paper, with an action zone for each role in the process.

Most teams' members know about the problems in their current, undefined way of doing work. In most cases, it is possible to diagram the way the team wants the improved process to be, without diagramming fully the way work is being done now. Usually the diagramming will be a combination of the two, with lots of changes. Regardless of how you get there, the end result should be a process flow diagram that is logical, contains all needed steps, does not show duplicated work, does not have work being done without purpose, and is easy to
follow.

The Process Flow Diagram: Instructions

If there are more than five roles in the process, group several together in one action zone, and as you draw the diagram just note which role is performing a step in that grouped zone. Usually in cases where there are more than five departments in a process, two or more of them have significantly fewer steps than other departments. These are ideal candidates for grouping on the diagram.

Line the meeting room walls with self-stick flip-chart paper. Draw lines to separate the action zones, write the role names in the appropriate zones, on the far left of the diagram. Now set the stage for the team by describing the state of the process right at the start. Ask what happens now. Draw process steps in the appropriate action zones; if a step will involve making a decision that affects process flow, put a decision point (diamond) symbol and write the decision question inside it. Keep asking, "what happens now?" until the last step in the process has been drawn.

Ask for a volunteer to copy the diagram and distribute copies before the next meeting.

At the next meeting, the team needs to walk through the process with as much realism as possible, playing roles, to find areas that need to be changed. Once the team agrees that the diagram will work, this step is done. Be sure the team understands that this is not set in stone—that as definition work progresses, the diagram will probably be changed.

The diagram will be put into a file using an online drawing tool, and incorporated into the Process Definition Document before Step 8—Inspection Process.

Step 6—Document Process Steps, Measures, and Metrics

First, each step is documented. The Process Definition Team members determine if one team member will document all the steps, or if the work will be split among members. For each step, the following information needs to be documented in the step description:
Purpose of the Step

1. Roles and Responsibilities (If more than onerole is involved, who does what?)
2. Entry Criteria (How do you know you are supposed to be at thisstep?)
3. Inputs (What do you need in order to complete this step? Inputs include information and decisions.)

4. Next Step (Most of the time, the next step on the process flow diagram. If a decision point follows this step, document that here and tell where control will go based on the decision's value.)
5. Exit Criteria (How do you know you are really done with this step?)
6. Output (Outputs include decisions, information, communication, and products.)
7. Work Instructions (Exactly what to do, in order.)
8. Tools/Techniques (Are there any tools, forms, standards, or special techniques to use in following the work instructions? Where do I find them?)
9. Special Considerations (Are there exceptions to be considered for this step? Additional information you need the reader to know?)

Next, the team defines measures and metrics for the process, using the Goal/Question/Metric approach. A Process Metrics Plan is documented, which describes the overall metrics program and documents each selected metric in detail.

After metrics have been identified, documented, and selected, the step descriptions are reviewed to determine where measurement collection activities need to be in place, and these activities are added to the appropriate step descriptions.

The Facilitator needs to know metrics and measurement theory, and be well versed in the psychological factors that can derail a metrics program.

Recommended Metrics Planning Approach

"Managing Metrics Is Not an Oxymoron," Douglas B. Hamilton, published in the proceedings from the Seventh International Conference on Software Quality.

Step 7—Acquire or Develop Forms, Tools, Standards, Examples

The Process Definition Team examines each Step Description and determines if there is any need for a standard, tool, checklist, or template in completing the work described in the step. For example, for Step 5—Document Global Process Information, Diagram the Process Flow, two templates would make the work easier. We need a word processing template, "Process Definition Document" for documenting the global process information and a template in a graphics tool to use in documenting process flow diagrams.

The team needs to identify, design or specify, and plan to acquire or develop all items needed to implement the process. Usually a process can be implemented in phases, so the team needs to decide which tools must be available for the first implementation phase, and which can be purchased or developed later.

Step 8—Formal Inspection Process

The Formal Inspection process, designed to find defects in software deliverables, is also the best way to find defects in any documentation throughout the organization. The team uses the formal Inspection process to find defects in the process documentation—the Process Definition Document, the Process Metrics Plan, and any forms, templates, and standards developed for the process. The Facilitator is responsible for leading the Inspection effort, unless the organization already has an Inspection process in place with trained Inspection Leaders. This step is complete when all items have successfully passed Inspection.

Recommended Reference

Software Inspection, Tom Gilb and Dorothy Graham, is an excellent reference.

Step 9—Plan and Execute Process Implementation

At this point, the Process Definition Team should have a rather complete list of Implementation Issues on their list. A plan with a list of implementation and process management activities, estimated effort, and target dates is now documented and reviewed with the Sponsor. After the plan is approved, the team puts it into action.

1.8 The Managed Process

A managed process is a performed process that is planned and executed in accordance with policy; employs skilled people having adequate resources to produce controlled outputs; involves relevant stakeholders; is monitored, controlled, and reviewed; and is evaluated for adherence to its process description.

The process can be instantiated by a project, group, or organizational function. Management of the process is concerned with institutionalization and the achievement of other specific objectives established for the process, such as cost, schedule, and quality objectives. The control provided by a managed process helps to ensure that the established process is retained during times of stress.

The requirements and objectives for the process are established by the organization. The status of the work products and services are visible to management at defined points (e.g., at major milestones, on completion of major tasks). Commitments are established among those who perform the work and the relevant stakeholders and are revised as necessary. Work products are reviewed with relevant stakeholders and are controlled. The work products and services satisfy their specified requirements.

A critical distinction between a *performed process* and a *managed process* is the extent to which the process is managed. A managed process is planned (the plan can be part of a more encompassing plan) and the execution of the process is managed against the plan. Corrective actions are taken when the actual results and execution deviate significantly from the plan. A managed process achieves the objectives of the plan and is institutionalized for consistent execution.

1.9 The Optimization Process

The purpose of **optimization** is to achieve the "best" design relative to a set of prioritized criteria or constraints. These include maximizing factors such as productivity, strength, reliability, longevity, efficiency, and utilization. ... This decision-making process is known as **optimization**.

What is the purpose of optimization?

Optimization problem: Maximizing or minimizing some function relative to some set, often representing a range of choices available in a certain situation. The function allows comparison of the different choices for determining which might be "best." Common applications: Minimal cost, maximal profit, minimal error, optimal design, optimal management, variational principles.

Goals of the subject

The understanding of

Modeling issues—

What to look for in setting up an optimization problem?
What features are advantageous or disadvantageous?
What devices/tricks of formulation are available?
How can problems usefully be categorized?

Analysis of solutions—

What is meant by a "solution?"
When do solutions exist, and when are they unique?
How can solutions be recognized and characterized?
What happens to solutions under perturbations?

Numerical methods—

How can solutions be determined by iterative schemes of computation?
What modes of local simplification of a problem are convenient/ appropriate?
How can different solution techniques be compared and evaluated?

Distinguishing features of optimization as a mathematical discipline:

Descriptive −→ Prescriptive
Equations −→ Inequalities
Linear/Nonlinear −→ Convex/Nonconvex
Differential Calculus −→ Sub Differential Calculus

Finite-dimensional optimization:

The case where a choice corresponds to selecting the values of a finite number of real variables, called decision variables. For general purposes the decision variables may be denoted by x1, . . . , xn and each possible choice therefore identified with a point x = (x1, . . . , xn) in the space IRn . This is what we'll be focusing on in this course.

Feasible set: The subset C of IRn representing the allowable choices x = (x1, . . . , xn). Objective function: The function f0(x) = f0(x1, . . . , xn) that is to be maximized or minimized over C.

Constraints: Side conditions that are used to specify the feasible set C within IR^n .

Equality constraints: Conditions of the form $f_i(x) = c_i$ for certain functions f_i on IR^n and constants c_i in IR^n

Inequality constraints: Conditions of the form fi(x) ≤ ci or fi(x) ≥ ci for certain functions fi on IRn and constants ci in IR.

Range constraints: Conditions restricting the values of some decision variables to lie within certain closed intervals of IR. Very important in many situations, for instance, are nonnegativity constraints: some variables xj may only be allowed to take values ≥ 0; the

interval then is $[0,\infty)$. Range constraints can also arise from the desire to keep a variable between certain upper and lower bounds.

Linear constraints: Range constraints or conditions of the form $f_i(x) = c_i$, $f_i(x) \leq c_i$, or $f_i(x) \geq c_i$, in which the function is linear in the standard sense of being expressible as sum of constant coefficients times the variables $x_1, \ldots, x_n$.

Data parameters: General problem statements usually involve not only decision variables but symbols designating known coefficients, constants, or other data elements. Conditions on such elements, such as the nonnegativity of a particular coefficient, are not among the "constraints" in a problem of optimization, since the numbers in question are supposed to be given and aren't subject to choice.

Mathematical programming: A traditional synonym for finite-dimensional optimization. This usage predates "computer programming," which actually arose from early attempts at solving optimization problems on computers. "Programming," with the meaning of optimization, survives in problem classifications such as linear programming, quadratic programming, convex programming, integer programming, etc.

1.10 Process Reference Models

1.11 Capability Maturity Model (CMM)

The Capability Maturity Model (CMM) is a methodology used to develop and refine an organization's software development process. The model describes a five-level evolutionary path of increasingly organized and systematically more mature processes. CMM was developed and is promoted by the Software Engineering Institute (SEI), a research and development center sponsored by the U.S. Department of Defense (DoD). SEI was founded in 1984 to address software engineering issues and, in a broad sense, to advance software engineering methodologies. More specifically, SEI was established to optimize the process of developing, acquiring, and maintaining heavily software-reliant systems for the DoD. Because the processes involved are equally applicable to the software industry as a whole, SEI advocates industry-wide adoption of the CMM.

The CMM is similar to ISO 9001, one of the ISO 9000 series of standards specified by the International Organization for Standardization (ISO). The ISO 9000 standards specify an effective quality system for manufacturing and service industries; ISO 9001 deals specifically with software development and maintenance. The main difference between the two systems lies in their respective purposes: ISO 9001 specifies a minimal acceptable quality level for software processes, while the CMM establishes a framework for continuous process improvement and is more explicit than the ISO standard in defining the means to be employed to that end.

Levels of CMM

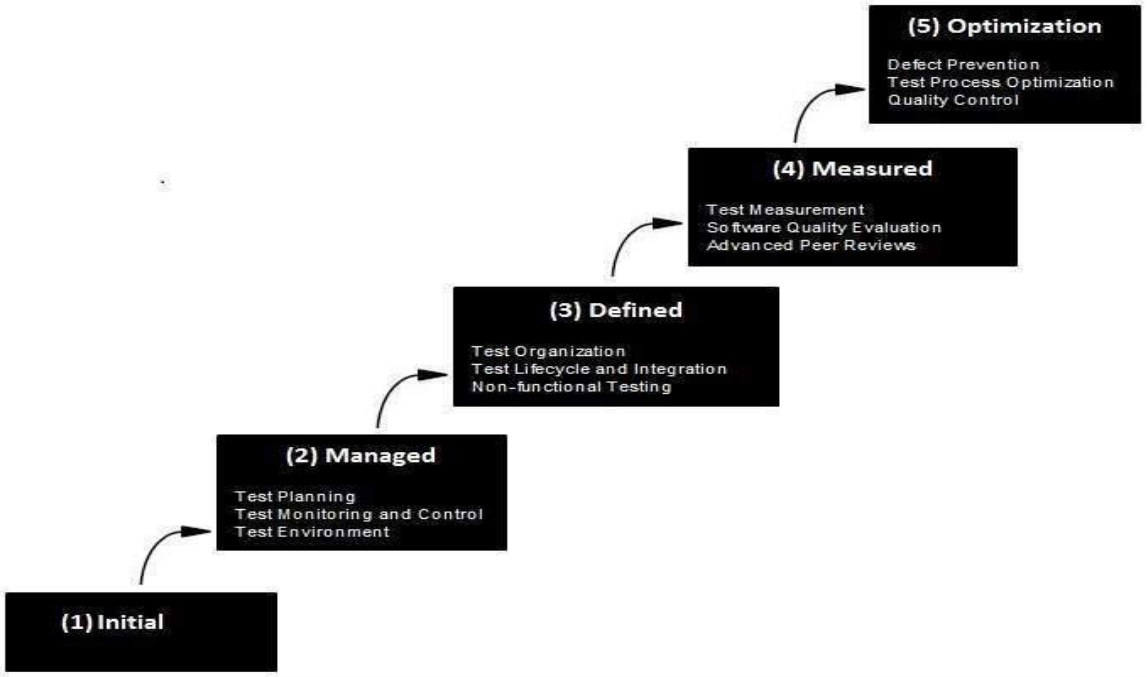


Figure 3: CMM Reference Model

- **Level One :Initial** - The software process is characterized as inconsistent, and occasionally even chaotic. Defined processes and standard practices that exist are abandoned during a crisis. Success of the organization majorly depends on an individual effort, talent, and heroics. The heroes eventually move on to other organizations taking their wealth of knowledge or lessons learnt with them.

- **Level Two: Repeatable** - This level of Software Development Organization has a basic and consistent project management processes to track cost, schedule, and functionality. The process is in place to repeat the earlier successes on projects with similar applications. Program management is a key characteristic of a level two organization.

- **Level Three: Defined** - The software process for both management and engineering activities are documented, standardized, and integrated into a standard software process for the entire organization and all projects across the organization use an approved, tailored version of the organization's standard software process for developing, testing and maintaining the application.

- **Level Four: Managed** - Management can effectively control the software development effort using precise measurements. At this level, organization set a quantitative quality goal for both software process and software maintenance. At this maturity level, the performance of processes is controlled using statistical and other quantitative techniques, and is quantitatively predictable.

- **Level Five: Optimizing** - The Key characteristic of this level is focusing on continually improving process performance through both incremental and innovative technological improvements. At this level, changes to the process are to improve the process performance and at the same time maintaining statistical probability to achieve the established quantitative process-improvement objectives.

1.12 Capability Maturity Model Integration

The Capability Maturity Model Integration (CMMI) is a process and behavioral model that helps organizations streamline process improvement and encourage productive, efficient behaviors that decrease risks in software, product and service development.

The CMMI was developed by the Software Engineering Institute at Carnegie Mellon University as a process improvement tool for projects, divisions or organizations. The DoD and U.S. Government helped develop the CMMI, which is a common requirement for DoD and U.S. Government software development contracts. The CMMI is currently administered by the CMMI Institute, which was purchased by the ISACA in 2016.

CMMI model

The CMMI starts with an appraisal process that evaluates three specific areas: process and service development, service establishment and management, and product and service acquisition. It's designed to help improve performance by providing businesses with everything they need to consistently develop better products and services.

But the CMMI is more than a process model; it's also a behavioral model. Businesses can use the CMMI to tackle the logistics of improving performance by developing measurable benchmarks, but it can also create a structure for encouraging productive, efficient behavior throughout the organization.

Standard CMMI Appraisal Method For Process Improvement (Scampi)

SCAMPI is the official appraisal method used by the CMMI institute. It is outlined in the SCAMPI Method Definition Document, which is included in the CMMI appraisal reference documents. There are three appraisal classes: Class A, B and C.

Scampi A

The most rigorous appraisal method, SCAMPI A is most useful after multiple processes have been implemented. It provides a benchmark for businesses and is the only level that results in an official rating. It must be performed by a certified lead appraiser, who is part of the on-site appraisal team.

Scampi B

This appraisal is less formal than SCAMPI A; it helps find a target CMMI maturity level, predict success for evaluated practices and give the business a better idea of where they stand in the maturity process.

Scampi C

This appraisal method is shorter, more flexible and less expensive than Class A or B. It's designed to quickly assess a business's established practices and how those will integrate or align with CMMI practices. It can be used at a high-level or micro-level, to address organizational issues or smaller process or departmental issues. It involves more risk than Class A or B, but it's more cost-effective.

CMMI Maturity Levels

The CMMI model breaks down organizational maturity into five levels. For businesses that embrace CMMI, the goal is to raise the organization up to Level 5, the "optimizing" maturity level. Once businesses reach this level, they aren't done with the CMMI. Instead, they focus on maintenance and regular improvements.

CMMI's five Maturity Levels are:
Initial

Processes are viewed as unpredictable and reactive. At this stage, "work gets completed but it's often delayed and over budget." This is the worst stage a business can find itself in — an unpredictable environment that increases risk and inefficiency.

Managed

There's a level of project management achieved. Projects are "planned, performed, measured and controlled" at this level, but there are still a lot of issues to address.

Defined

At this stage, organizations are more proactive than reactive. There's a set of "organization-wide standards" to "provide guidance across projects, programs and portfolios." Businesses understand their shortcomings, how to address them and what the goal is for improvement.

Quantitatively managed

This stage is more measured and controlled. The organization is working off quantitative data to determine predictable processes that align with stakeholder needs. The business is ahead of risks, with more data-driven insight into process deficiencies.
Optimizing: Here, an organization's processes are stable and flexible. At this final stage, an organization will be in constant state of improving and responding to changes or other opportunities. The organization is stable, which allows for more "agility and innovation," in a predictable environment.

Once organizations hit Levels 4 and 5, they are considered high maturity, where they are "continuously evolving, adapting and growing to meet the needs of stakeholders and customers." That is the goal of the CMMI: To create reliable environments, where products, services and departments are proactive, efficient and productive.

Maturity Levels And Process Areas

Here is a list of all the corresponding process areas defined for a S/W organization. These process areas may be different for different organization.

This section is just giving names of the related process areas, for more detail about these Process Areas go through CMMI Process Areas Chapter.

Table 1: Maturity Level and Process Areas

Level	Focus	Key Process Area	Result
5 Optimizing	Continuous Process Improvement	• Organizational Innovation and Deployment • Causal Analysis and Resolution	Highest Quality / Lowest Risk
4 Quantitatively Managed	Quantitatively Managed	• Organizational Process Performance • Quantitative Project Management	Higher Quality / Lower Risk
3 Defined	Process Standardization	• Requirements Development • Technical Solution • Product Integration • Verification • Validation • Organizational Process Focus • Organizational Process Definition • Organizational Training • Integrated Project Mgmt (with IPPD extras) • Risk Management • Decision Analysis and Resolution • Integrated Teaming (IPPD only) • Org. Environment	Medium Quality / Medium Risk

ST.MARTIN'S ENGINEERING COLLEGE
TECHNOLOGY FOR PROSPERITY

		for Integration (IPPD only) • Integrated Supplier Management (SS only)	

2 Managed	Basic Project Management	• Requirements Management • Project Planning • Project Monitoring and Control • Supplier Agreement Management • Measurement and Analysis • Process and Product Quality Assurance • Configuration Management	Low Quality / High Risk
1 Initial	Process is informal and Adhoc		Lowest Quality / Highest Risk

1.13 People Capability Maturity Model (PCMM)

People Capability Maturity Model (PCMM) is a maturity framework that focuses on continuously improving the management and development of the human assets of a software or information systems organization. PCMM can be perceived as the application of the principles of Capability Maturity Model to human assets of a software organization. It describes an evolutionary improvement path from ad hoc, inconsistently performed practices, to a mature, disciplined, and continuously improving development of the knowledge, skills, and motivation of the workforce. Although the focus in People CMM is on software or information system organizations, the processes and practices are applicable for any organization that aims to improve the capability of its workforce. PCMM will be guiding and effective particularly for organizations whose core processes are knowledge intensive.

The primary objective of the People Capability Maturity Model is to improve the capability of the entire workforce. This can be defined as the level of knowledge, skills, and

process abilities available for performing an organization's current and future business activities.

10 Principles of People Capability Maturity Model (PCMM)

The People Capability Maturity Model describes an evolutionary improvement path from ad hoc, inconsistently performed workforce practices, to a mature infrastructure of practices for continuously elevating workforce capability. The philosophy implicit the PCMM can be summarized in ten principles.

1. In mature organizations, workforce capability is directly related to business performance.
2. Workforce capability is a competitive issue and a source of strategic advantage.
3. Workforce capability must be defined in relation to the organization's strategic business objectives.
4. Knowledge-intense work shifts the focus from job elements to workforce competencies.
5. Capability can be measured and improved at multiple levels, including individuals, workgroups, workforce competencies, and the organization.
6. An organization should invest in improving the capability of those workforce competencies that are critical to its core competency as a business.
7. Operational management is responsible for the capability of the workforce.
8. The improvement of workforce capability can be pursued as a process composed from proven practices and procedures.
9. The organization is responsible for providing improvement opportunities, while individuals are responsible for taking advantage of them.
10. Since technologies and organizational forms evolve rapidly, organizations must continually evolve their workforce practices and develop new workforce competencies.

The People Capability Maturity Model (People CMM) is a roadmap for implementing workforce practices that continuously improve the capability of an organization's workforce. Since an organization cannot implement all of the best workforce practices in an afternoon, the People CMM introduces them in stages. Each progressive level of the People CMM produces a unique transformation in the organization's culture by equipping it with more powerful practices for attracting, developing, organizing, motivating, and retaining its workforce. Thus, the People CMM establishes an integrated system of workforce practices that matures through increasing alignment with the organization's business objectives, performance, and changing needs.

Although the People CMM has been designed primarily for application in knowledge intense organizations, with appropriate tailoring it can be applied in almost any organizational setting. The People CMM's primary objective is to improve the capability of the workforce. Workforce capability can be defined as the level of knowledge, skills, and process abilities available for performing an organization's business activities.

People Capability Maturity Model (PCMM) Levels

The People Capability Maturity Model consists of five maturity levels. Each maturity level is an evolutionary plateau at which one or more domains of the organization's processes are transformed to achieve a new level of organizational capability. The five levels of People CMM are defined as follows:



Figure 4: PCCM Levels

Table 2: PCMM Threads

Levels	People CMM Threads			
	Developing competency	Building workgroups& culture	Motivating & managing performance	Shaping the workforce
5 Optimizing	Continuous Capability Improvement		Organisational Performance Alignment	Continuous Workforce Innovation
4 Predictable	Competency Based Assets Mentoring	Competency Integration Empowered workgroups	Quantitative Performance Management	Organisational Capability Management
3 Defined	Competency development Competency Analysis	Workgroup Development Participatory Culture	Competency Based Practices Career Development	Workforce Planning
2 Managed	Training and Development	Communication & Coordination	Compensation Performance Management Work Environment	Staffing

1.14 Personal Software Process

The PSP is a personal process for developing software or for doing any other defined activity. The PSP includes
defined steps
forms
standards

It provides a measurement and analysis framework for characterizing and managing your personal work.

It is also a defined procedure that helps you to improve your personal performance.
PSP Process Flow

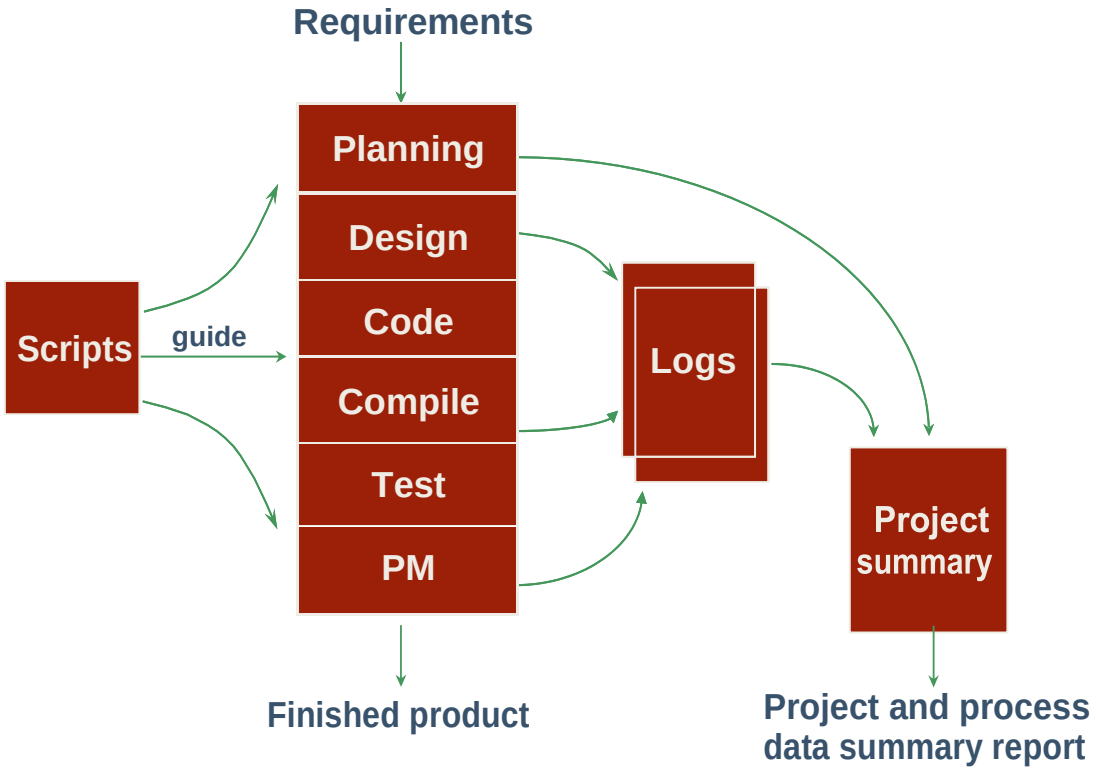


Figure 5: PSP Process Flow Diagram

The PSP process is designed for individual use.
It is based on scaled-down industrial software practice.
The PSP course demonstrates the value of using a defined and measured process.
It helps you and your organization meet the increasing demands for high quality and timely software.

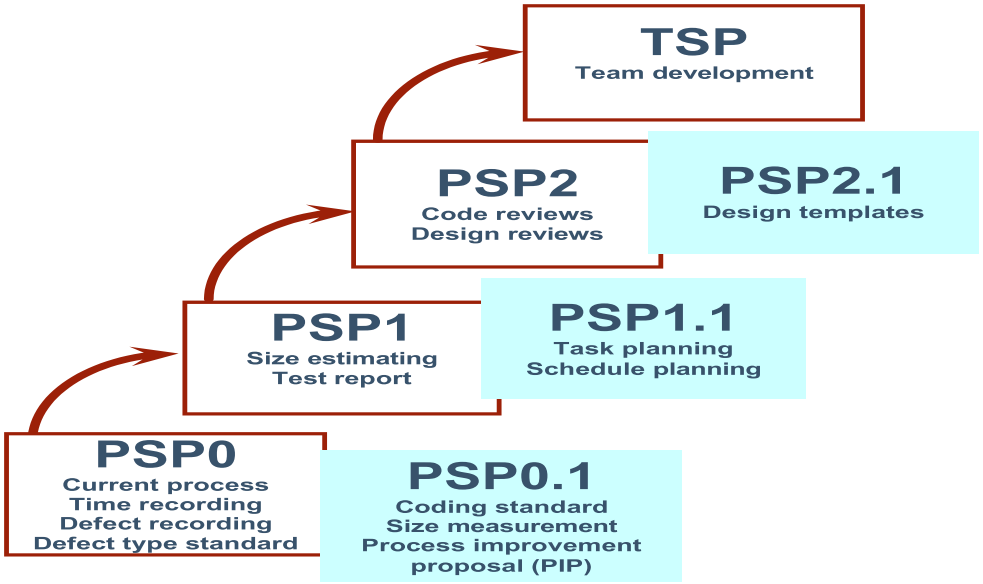


Figure 6: PSP Phases Diagram

Table 3: PSP Phases

Phase	Emphasis	Features
0	Personal Management	Current process plus basic measures: development time, defects injected and removed; process: planning, development, analysis
0.1		Coding standards, process improvement proposal form, size measurements
1	Personal Planning	PROBE; Size estimation, time estimates, test report
1.1		Task planning, schedule planning
2	Personal Quality	Defect management: code reviews, design reviews
2.1		Design specification and analysis; defect prevention; process analysis; process benchmarks
3	Scaling Up	Cyclic development

Learning PSP 0

- Personal measurement
- Forms and scripts
- Time, defects injected and removed
- Phases: planning, development, postmortem
- PSP0.1: add in coding standards, size measurement, and process improvement proposal

Learning PSP 1

- Personal planning
- PROBE estimation; confidence intervals
- PSP1.1: schedule and task planning

Table 4: PSP1 Process Script (SEI)

Phase Number	Purpose	To guide you in developing module-level programs
	Entry Criteria	• Problem description • PSP1 Project Plan Summary form • ***Size Estimating Template*** • ***Historical estimate and actual size data*** • Time and Defect Recording Logs • Defect Type Standard • Stop watch (optional)
1	Planning	• Produce or obtain a requirements statement. • ***Use the PROBE method to*** estimate the total new and changed LOC required. • ***Complete the Size Estimate Template.*** • Estimate the required development time. • Enter the plan data in the Project Plan Summary form. • Complete the Time Recording Log.
2	Development	• Design the program. • Implement the design. • Compile the program and fix and log all defects found. • Test the program and fix and log all defects found. • Complete the Time Recording Log.
3	Postmortem	Complete the Project Plan Summary form with actual time, defect, and size data.
	Exit Criteria	• A thoroughly tested program • Completed Project Plan Summary form with estimated and actual data • ***Completed Size Estimating Template*** • ***Completed Test Report Template*** • Completed PIP forms • Completed Defect and Time Recording Logs

Learn PSP 2

- Personal quality
- Defect management: data, review checklists
- PSP2.1: design specification, defect prevention, process analysis, process benchmarks
-

Learn PSP 3:

- Scaling up
- Cyclic development
- Design verification; process definition principles
- Subsumed by TSP

Overall PSP Strategy

1. Gather data
2. Estimate and plan
3. Manage defects
4. Manage yield
5. Control cost of quality

Gather Data

- **Measurements taken**
  - Time in each process activity (and for interrupts)
  - Defects introduced and removed for each activity
  - Developed product size (LOC)
    - Base, added, modified, deleted, new and changed, reused, new reuse, total
- **Metrics computed**
  - Size and time estimating error
  - Cost-performance index
  - Defect
    - Injected and removed per hour
    - Density
  - Process yield
  - Appraisal and failure cost of quality
  - Appraisal to failure ratio
  -

Estimate and Plan

PROBE - proxy based estimation method
PSP proxies: functions and object
Others include function points, screens, reports, sections of text
Linear regression on at least 3 prior projects
Goal is to improve estimates over time
PSP students improved their size estimates from 31% (within 20%) to 42% between programs one and ten
Improved time estimates from 33% (within 20%) to 49%

SEI

- **Base** When an existing product is enhanced, base LOC is the size of the original product version before any modifications are made.
- **Added** Code written for a new program or added to an existing base program.
- **Modified** LOC in an existing (Base) program that are changed.
- **Deleted** LOC in an existing (Base) program that are deleted.
- **New and Changed** When engineers develop software, it takes them much more time to add or modify a LOC than it does to delete or reuse one. Thus, in the PSP, engineers use only the **Added** or **Modified** code to make size and resource estimates. This code is called the **New and Changed** LOC.
- **Reused** Code taken from a reuse library and used, without modification, in a new program. Reuse does not count the unmodified base code retained from a prior program version and it does not count any code that is reused with modifications.
- **New Reuse** LOC that an engineer develops and contributes to the reuse library.
- **Total** Total size of a program, regardless of its source (= **Base - Deleted** + **Added** + **Reuse**).
- Record, for each defect
  - Activity (phase) during which defect was injected and removed
    - Planning, design, design review, code, code review, compile, test

  - Defect type (next slide)
  - Fix time
  - Description
- Students reduced defect rates from 116/KLOC to 49/KLOC between programs one and ten
  - Standard deviation also reduced

Table 5: Defect Types

Type Number	Type Name	Description
10	Documentation	comments, messages
20	Syntax	spelling, punctuation, types, instruction formats
30	Build, package	change management, library, version control
40	Assignment	declaration, duplicate name, scope, limits
50	Interface	procedure calls and references, I/O, user format
60	Checking	error messages, inadequate checks
70	Data	structure, content
80	Function	logic, pointers, loops, recursion, computations, function defects
90	System	configuration, timing, memory
100	Environment	design, compile, test, or other support-system problems

Manage Yield

- Yield is PSP's principle quality measure
- If it is costly to find a defect during testing, then you need to find it earlier (during review)
  - (Or not insert it in the first place)
- Hold review before compilation
  - (But aren't compilers cheaper than programmers?)
  - (And desk check every new compilation)
- *Yield:* % defects found and fixed before compilation
  - Engineers review code before first compile
  - 9% of "syntax" error get by compiler
  - Defects found at compile time correlate with defects found during test (r = .71)
  - Strong correlation between defects found during test and customer failures (r = .91)
- Introduction of design and code reviews strongly improves yield

Control Cost of Quality

- Appraisal cost
  - Time spent in design and code reviews
- Failure cost
  - Time spent in compile and test
- Prevention costs
  - Prototyping, formal specification
  - Not part of PSP
- Appraisal to failure ratio (A/FR)
  - Raise until quality is sufficient then gradually lower
  - Initial target at least two

PSP Benefits

- Increases personal commitment by investing each engineer with process responsibility
- Assists engineers in making accurate plans
- Provides steps engineers can take to improve personal and project quality
- Sets benchmarks to measure personal process improvements
- Demonstrates the impact of process changes on an engineer's performance

Team Software Process (Tsp)

In combination with the personal software process (PSP), the **team software process** (**TSP**) provides a defined operational process framework that is designed to help teams of managers and engineers organize projects and produce software the principles products that range in size from small projects of several thousand lines of code (KLOC) to very large projects greater than half a million lines of code. The TSP is intended to improve the levels of quality and productivity of a team's software development project, in order to help them better meet the cost and schedule commitments of developing a software system.

The initial version of the TSP was developed and piloted by Watts Humphrey in the late 1990s and the Technical Report for TSP sponsored by the U.S. Department of Defense was published in November 2000. The book by Watts Humphrey, *Introduction to the Team Software Process*, presents a view of the TSP intended for use in academic settings, that focuses on the process of building a software production team, establishing team goals, distributing team roles, and other teamwork-related activities.

TSP Strategy

- ★ Provide a simple process framework based on the PSP.
- ★ Use modest, well-defined problems.
- ★ Develop products in several cycles.
- ★ Establish standard measures for quality and performance.
- ★ Provide detailed role definitions.
- ★ Use role and team evaluations.
- ★ Require process discipline.
- ★ Provide guidance on teamwork problems.

1.15 TSP Process

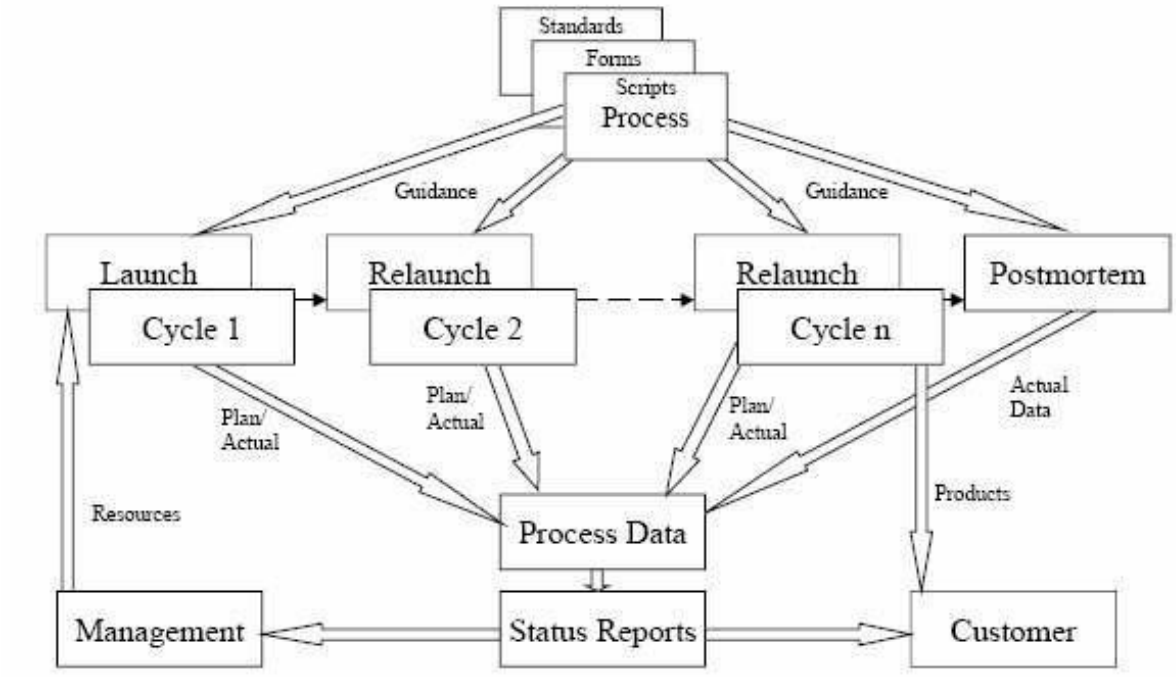


Figure 7: TSP Process Flow

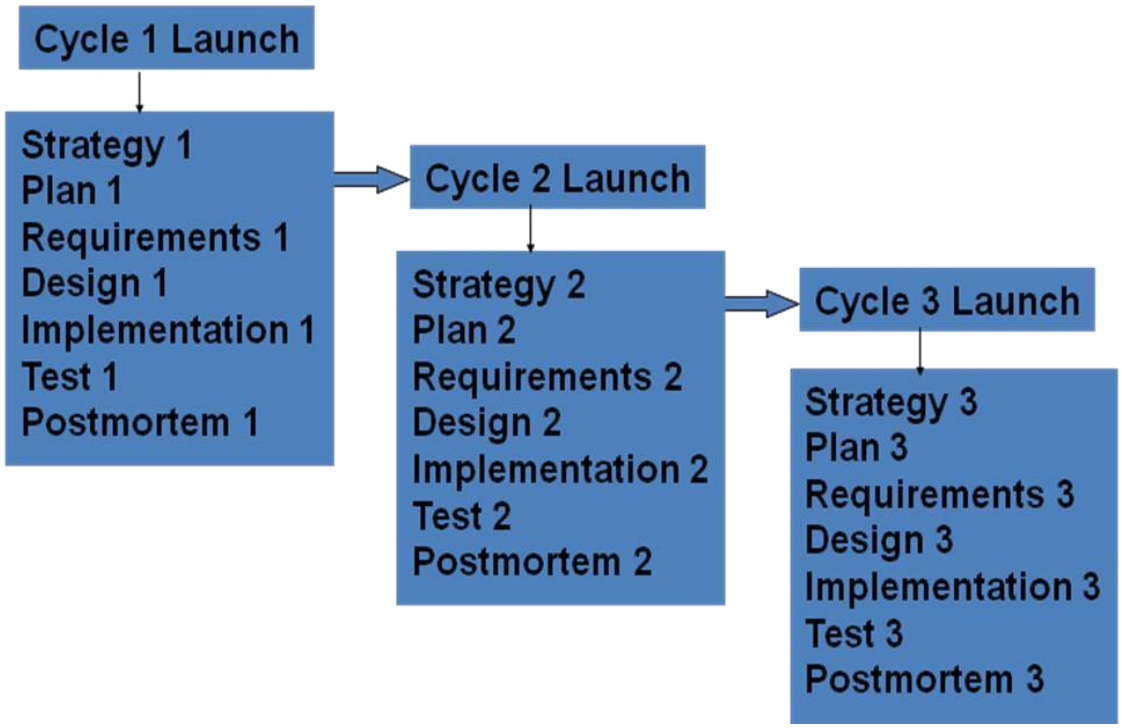


Figure 8: TSP Cycle Diagram

Life Cycle Phases

- Launch
- Strategy
- Plan
- Requirements
- Design
- Implement
- Test
- Postmortem

- **Launch**

  - review course objectives
  - describe the TSP structure and content
  - assign student teams and roles
  - describe the customer needs statement
  - establish team and individual goals

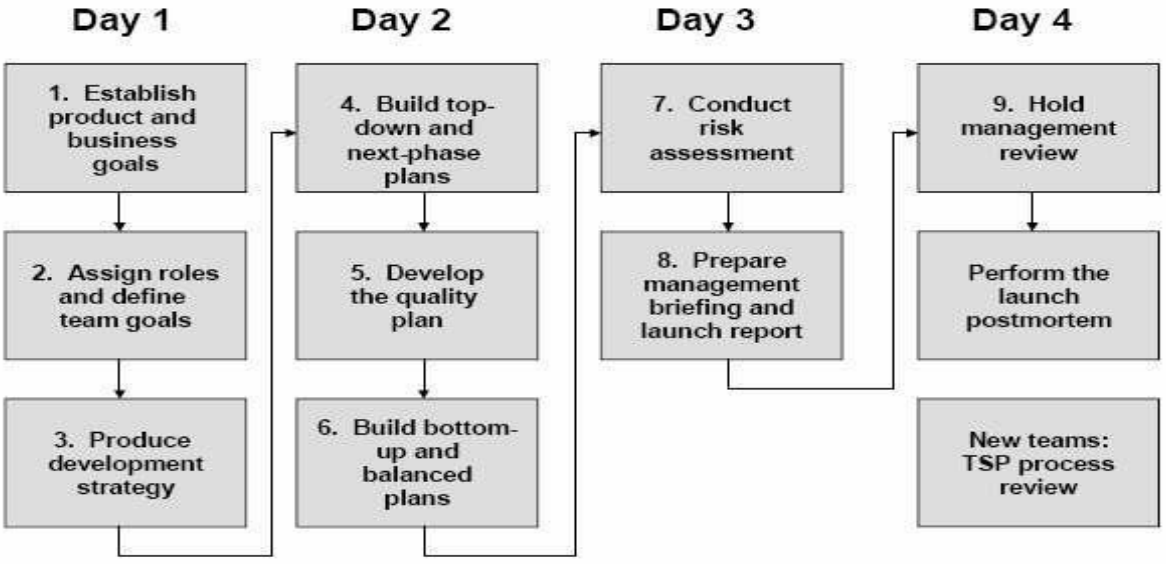


Figure 9: Launch Process

- **Strategy**

  - create a conceptual design for the product
  - establish the development strategy: decide what will be produced in each cycle
  - make initial size and effort estimates
  - establish a configuration management plan
  - Reuse plan
  - Risk Management

- **Planning**

- estimate the size of each artifact to be developed (SRS, SDS, code)

- identify tasks to be performed; estimate time to complete each task; assign tasks to team members
- make a weekly schedule for task completion
- make a quality plan

- **Requirements**

  - analyze need statement and interview customer
  - specify the requirements
  - inspect the requirements
  - develop a system test plan

- **Design**

  - create a high-level design
  - specify the design
  - inspect the design
  - develop an integration test plan

- **Implementation**

  - use the PSP to implement modules/units
  - create detailed design of modules/units
  - review the design
  - translate the design to code
  - review the code
  - compile and test the modules/units
  - analyze the quality of the modules/units

- **Test**

  - build and integrate the system
  - conduct a system test
  - produce user documentation

- **Postmortem**

  - conduct a postmortem analysis
  - write a cycle report
  - produce peer and team evaluations

Team (Dyer)

A team consists of at least two people.

- The members are working toward a common goal.
- Each person has a specific assigned role.
- Completion of the mission requires some form of dependency among the group members.

Effective Teams

- The members are skilled.
- The team's goal is important, defined, visible, and realistic.
- The team's resources are adequate for the job.
- The members are motivated and committed to meeting the team's goal.
- The members cooperate and support each other.
- The members are disciplined in their work.

Team Building

- The team members establish common goals and defined roles.
- The team develops an agreed-upon strategy.
- The team members define a common process for their work.
- All team members participate in producing the plan, and each member knows his or her personal role in that plan.
- The team negotiates the plan with management.
- Management reviews and accepts the negotiated plan.
- The team members do the job in the way that they have planned to do it.
- The team members communicate freely and often.
- The team forms a cohesive group: the members cooperate, and they are all committed to meeting the goal.
- The engineers know their status, get feedback on their work, and have leadership that sustains their motivation.

Selecting Roles

- Team Leader
- Development Manager
- Planning Manager
- Quality/Process Manager
- Support Manager
- Customer interface manager
- Design manager
- Test manager
- Safety manager
- Security manager
- Performance manager

Team Leader Responsibilities

- Motivating team members
- Handling customer issues
- Interaction with management
- Day-to-day direction of the work
- Protecting team resources
- Resolving team issues
- Conducting team meetings
- Reporting on the work status

Development Manager

• Leads and guides the team in designing and developing the product
  – Lead the team in producing the development strategy and the product conceptual design
  – Lead the team in producing the design specification (SDS)
• If there is no separate Design Manager or Software Architect
  – Lead the team in implementing the product

Planning Manager

• Supports and guides the team in planning and tracking their work
  – Lead the team in producing the task plan and schedule for each development cycle
  – Lead the team in producing the balanced team development plan
  – Track the team's progress against their plan

Quality / Process Manager

- Supports the team in defining their process needs, in making the quality plan and in tracking process and product quality
  – Lead the team in producing and tracking their quality plan
  – Identify where quality performance falls short of objectives.
  – Lead the team in defining, documenting, and maintaining their processes and development standards
  – Act as moderator and lead all team reviews and inspections

Support Manager

• Supports the team in determining, obtaining, and managing the tools needed to meet its technology and administrative support needs
  – Lead the team in determining their support needs and obtaining the needed tools and facilities
  – Lead the development and management of Change/Configuration Management System
  – Handle the team's issue and risk tracking system
  – Act as the team's reuse advocate

Task Planning

• Generate a default task list
• Modify the default list as appropriate and estimate sizes and times for each task
• Assign portions of tasks to individual engineers
• Determine total time required for the project cycle

Schedule Planning

• Use LOC estimates and LOC/hour rates for time estimation
• Generate TASK and SCHEDULE plans
  – Estimate times based upon LOC and LOC/hour estimates
  – Estimate time available for the team

Produce Size Estimates

- Produce the conceptual design
  - Objects for all cycles
- Select a development strategy
  - Allocate objects to cycles
  - Produce a minimal working subset in the first cycle
  - Development Manager leads

- Produce preliminary size estimates
  - Estimate LOC in each class
  - Use size estimates as a basis for allocation of tasks to cycles
  - Enter estimates on the STRAT form

Quality Planning

- Enter default quality criteria into the SUMQ form
  - the Percent Defect Free components in each of the defect detection phases
  - the defect removal yields in various phases (Phase Yields)
  - the rate at which defects are injected in various phases (Defect Injection Rates)

TSP Quality Guidelines

- Percent (of modules) Defect Free (PDF) at entrance to
  - Compile > 10%
  - Unit Test > 50%
  - Integration Test > 70%
  - System Test > 90%

- Defects/KLOC:
  - Total defects injected 75 - 150; If not PSP trained, use 100 to 200.
  - Compile < 10
  - Unit Test < 5
  - Integration Test < 0.5
  - System Test < 0.2

- Defect Ratios
  - Detailed design review defects /unit test defects > 2.0
  - Code review defects/compile defects > 2.0

The Yield Measure

Phase yield is the percentage of defects entering and injected in a phase that are removed in that phase

Process yield is the yield of all phases up to that point in the process

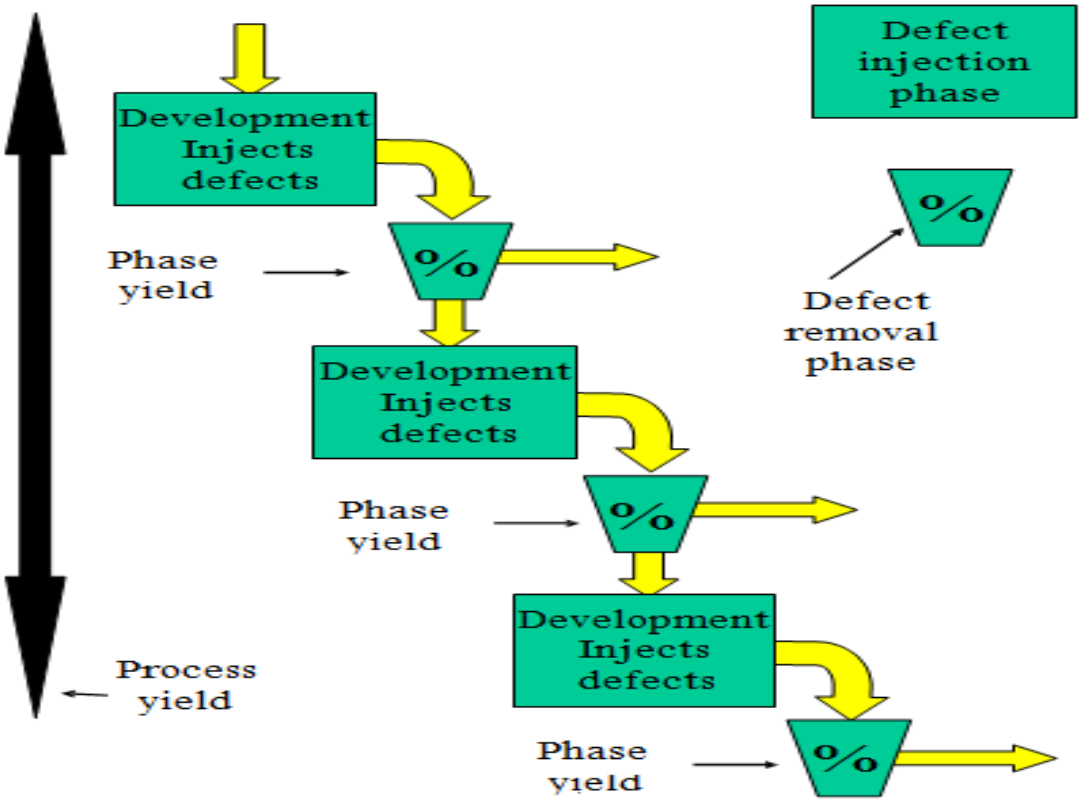


Figure 10: Yield Process

Phase Yields

- Team requirements inspections ~ 70%
- Design reviews and inspections ~ 70%
- Code reviews and inspections ~ 70%
- Compiling ~ 50% (90+ % of syntax defects)
- Unit test - at 5 or less defects/KLOC ~ 90%
  - For high defects/KLOC - 50-75%
- Integration and system test - at < 1.0 defects/KLOC ~ 80%
  - For high defects/KLOC - 30-65%
- Before compile >75%
- Before unit test > 85%
- Before integration test > 97.5%
- Before system test > 99%